

Improved Radix-based Approximate Homomorphic Encryption for Large Integers via Lightweight Bootstrapped Digit Carry

Gyeongwon Cha¹, Dongjin Park¹, and Joon-Woo Lee^{1*}

Department of Computer Science and Engineering, Chung-Ang University, Seoul,
Republic of Korea

{dbf1dk20, thrudgelmir, jwlee2815}@cau.ac.kr

Abstract. Homomorphic encryption (HE) for high-precision integers has been steadily researched through various schemes; however, these approaches incurred severe overhead as the bit-width grew, requiring larger parameters to support integers of several hundred to a thousand bits. A significant breakthrough was recently made by Boneh and kim (Crypto’25). Their scheme constructs a residue number system from the different slots of a single CKKS ciphertext. This enables arithmetic on thousand-bit integers without increasing parameters. However, RNS approach in Boneh et al., which performs approximate reduction, fundamentally cannot support non-arithmetic operations. Alternatively, radix-based approach proposed by Kim (CHES’25) can perform non-arithmetic operations, but they require $O(k)$ bootstraps for a bit-width k . This makes them highly inefficient, and thus impractical, for non-arithmetic operations requiring thousand-bit precision. This paper proposes an improved radix-based CKKS scheme, centered on a 2-step algorithm that optimizes the number of bootstraps required for the digit carry operation to $O(\log k)$. The proposed scheme requires only 3-6 bootstraps to restore the result of a 32-2048 bit integer multiplication to its unique representation, which enables the efficient implementation of non-arithmetic operations such as comparison. Furthermore, our scheme extends the radix-based system, previously limited to prime-power moduli, to support an efficient homomorphic reduction algorithm for arbitrary moduli. Furthermore, our experiments demonstrate substantial efficiency gains compared to Boneh et al. For example, for moduli used in homomorphic signatures (Curve25519, P-384, and 2048-bit RSA), our scheme can process up to $4\times$ more integers in a single ciphertext. Specifically for Curve25519, we also reduce the latency by $1.4\times$, shortening the amortized time by $5.6\times$ compared to Boneh et. al. and achieving a final processing time of 1.34 seconds per data point.

Keywords: Homomorphic Encryption · Discrete CKKS · High Precision Integer Arithmetic

* Corresponding author

1 Introduction

Homomorphic encryption (HE) [20] is a cryptographic scheme that allows arithmetic operations to be performed directly on encrypted data. Traditionally, selecting a HE scheme for integer data types involved a choice between two de facto standard schemes: DM/CGGI [14] and BFV/BGV [18,8,9]. DM/CGGI, a learning with errors (LWE) based scheme, excels at low-latency operations on single data point and supports a wide range of non-arithmetic functions via functional bootstrapping. In contrast, BFV/BGV, based on ring LWE, is highly effective for parallel processing over finite fields, making it well-suited for applications that process a large amount of data at once. As a result, the choice between the two schemes depended on the specific application: BFV/BGV was chosen when arithmetic operations on large amounts of data were required, while DM/CGGI was chosen for its functional flexibility with smaller batch sizes.

To resolve this trade-off, recent work has increasingly turned to the CKKS [13] scheme as a way to batch, or even entirely replace, operations on multiple DM/CGGI ciphertexts. Building on the work of [16], numerous follow-up studies have adapted the CKKS scheme—originally designed for approximate arithmetic on real or complex numbers—to handle discrete data such as integers. These variants, commonly known as *discrete CKKS*, have led to significant achievements, including integer operation [26,24,25,6] and CKKS-style (functional) bootstrapping [3,4,2,17].

Against this backdrop, this paper focuses on integer operation, because despite the recent achievements of discrete CKKS, a clear "winner" has not yet emerged to replace DM/CGGI for all such tasks. This is because current approaches, including both DM/CGGI and various discrete CKKS methods, exhibit starkly different trade-offs, excelling in some scenarios while underperforming in others. To illustrate this, we now analyze three recent, state-of-the-art approaches.

The first, the work of [24], focuses on maximizing amortized time. This method decomposes an integer in $\mathbb{Z}_{B^k}$ into base B and stores each of the k digits in separate ciphertexts. This enables massive parallel processing by utilizing the full number of available slots in CKKS. However, this design comes with a significant trade-off: a high latency that is proportional to the bit-widths. To handle digit carries, every integer operation requires $O(k)$ bootstraps. Consequently, this approach is only practical for applications requiring a large amount of low-precision integer operation, making DM/CGGI a more suitable choice in many other scenarios.

Attempting to find a middle ground, a second approach from [25] focuses on balancing latency and amortized time. Similar to [24], it decomposes integers into base B , but with a key difference: all k digits are stored together within a single ciphertext. To perform multiplication in this approach, the author utilizes a technique where they perform polynomial arithmetic on a sub-ring by partially performing *SlotToCoefficients*, a subroutine of CKKS bootstrapping. The method employs an approximate digit carry for arithmetic operations, reducing the bootstrapping cost to $O(\log k)$. However, non-arithmetic operations,

	Arithmetic	Non-arithmetic	# Slot	Arbitrary precision	Arbitrary modulus
TFHE-rs [28]	$O(k)$	$O(k)$	1	▲	▲
[24]	$O(k)$	$O(k)$	$O(N)$	▲	▲
[25]	$O(\log k)$	$O(k)$	$O(N/k)$	✓	▲
[6]	$O(\log k)$	×	$O(N/k)$	✓	✓
Ours	$O(\log k)$	$O(\log k)$	$O(N/k)$	✓	✓

Table 1. Comparison of recent (fully) HE schemes for integer arithmetic. We analyze the number of bootstrappings required for arithmetic and non-arithmetic operations as a function of the bit-width k of the target precision. The number of slots (# Slot) is expressed in big- O notation with respect to the ring dimension N and bit-width k , except for TFHE-rs, which processes a single integer. The checkmarks (✓), crosses (×), and triangles (▲) indicate the availability of arbitrary precision and arbitrary modulus. The (▲) signifies that the feature is supported, but requires high latency.

which require exact carries, still incur an $O(k)$ bootstrapping overhead. In conclusion, while this work is a strong competitor to DM/CGGI for arithmetic-only applications, DM/CGGI remains more advantageous for use cases requiring non-arithmetic operations.

Taking a completely different path, the work of [6] introduces a scheme for arithmetic over arbitrary modulus. Unlike the previous methods, this research adopts a homomorphic residue number system (RNS) approach. The core mechanism involves constructing $\mathbb{Z}_{p_i}$ operations for a smooth modulus $p = \prod p_i$ and homomorphically adapting the base conversion technique from [21] (a more detailed discussion is provided in the related works). This design enables thousand-bit modulus arithmetic without increasing cryptographic parameters, a major achievement for applications like homomorphic signing.

However, the scheme’s reliance on approximate reduction, while key to its arithmetic efficiency, is also its critical flaw. Due to the absence of exact reduction and the structural limitations of RNS, the system cannot support non-arithmetic operations like comparison. Consequently, while [6] is effectively the only efficient scheme for arithmetic-only applications, especially at large bit-widths, it cannot replace DM/CGGI in scenarios that require non-arithmetic functionality.

The analysis of these prior works suggests the future direction for HE for high-precision integer arithmetic. This leads to a fundamental question: is it possible to adopt the radix decomposition method for non-arithmetic operations while reducing the complexity of digit carries? Our research provides an affirmative solution to this question.

1.1 Our Contribution

We propose a large-integer HE scheme built on CKKS, which uses a (non-mixed) radix-based digit decomposition to enable exact arithmetic over arbitrary moduli. Our approach supports a wide range of bit-widths and guarantees a unique

$\log(t)$	TFHE-rs [28]		Ours	
	Latency	Amortized time	Latency	Amortized time
64	19.6 sec	19.6 sec	40.3 sec	39.3 msec
128	77.5 sec	77.5 sec	57.4 sec	112 msec
256	309 sec	309 sec	74.8 sec	292 msec

Table 2. Performance comparison of TFHE-rs and our scheme by bit-widths, $\log(t)$

representation for each digit. This uniqueness is a key factor that enables extension to advanced operations such as comparison, bit-shifting, modular reduction and bit-wise operations combined with bit-decomposition [4]. Before detailing our contributions, Table 1 briefly compares our work to previous works.

Fast Digit Carry Algorithm A key achievement of our work is resolving the digit-carry bottleneck in prior radix-based schemes [24,25]. We reduce the complexity of the exact digit-carrying algorithm from requiring $O(k)$ bootstraps to $O(\log k)$ for a B^k modulus. Concretely, our scheme requires just 3–6 bootstraps across the entire 32- to 2048-bit range. The experimental results empirically demonstrate the efficiency gains of our proposed scheme. On a single-threaded environment, the proposed scheme processes 1024 64-bit integer multiplications in a time equivalent to that required by TFHE-rs for two 64-bit integer multiplications. Moreover, as the bit-width increases, the superiority of the proposed scheme over TFHE-rs in terms of latency becomes even more evident, as shown in Table 2.

Improved Amortized time for Arbitrary Modular Arithmetic Our scheme extends beyond its native B^k form to support arithmetic over arbitrary modulus. We achieve this with a homomorphic design of *Montgomery reduction*, a standard technique in high-performance cryptography that avoids costly divisions. Furthermore, we achieve additional optimization by generalizing the *folding method*, which is particularly effective for specific structures like generalized Mersenne numbers (GMNs). This design provides a strong performance advantage in applications requiring large modulus operations, such as homomorphic signing. As a result of our experiments, when compared to [6] in the same environment, our scheme processes $4\times$ more data in a single ciphertext, leading to up to a $5.6\times$ improvement in amortized time, as shown in Table 3.

1.2 Technical Overview

In our scheme, an integer $z \in \mathbb{Z}_{B^k}$ is decomposed into k digit in base- B , and each digit is stored in a distinct CKKS slot. This digit representation can be expressed as a polynomial $z(X)$ of degree $k - 1$, which evaluates to z when $X = B$. Accordingly, arithmetic multiplication should be realized as polynomial multiplication. However, CKKS arithmetic operations are applied element-wise

Scheme	Modulus	# Slot	Multiplication time	
			Latency	Amortized time
Ours	Curve25519	128	172 sec	1.34 sec
	P-384	64	213 sec	3.3 sec
	RSA2048	16	440 sec	27.5 sec
[6]	Curve25519	32	242 sec	7.56 sec
	P-384	32	245 sec	7.65 sec
	RSA2048	4	316 sec	79 sec

Table 3. Timing for homomorphic modular multiplication. The number of slots indicates how many integers can be processed within a single ciphertext. Curve25519 and P-384 are elliptic curves with prime moduli $p = 2^{255} - 19$ and $p = 2^{384} - 2^{128} - 2^{96} + 2^{32} - 1$, respectively. RSA-2048 refers to a 2048-bit modulus $N = pq$, where p and q are randomly sampled 1024-bit primes.

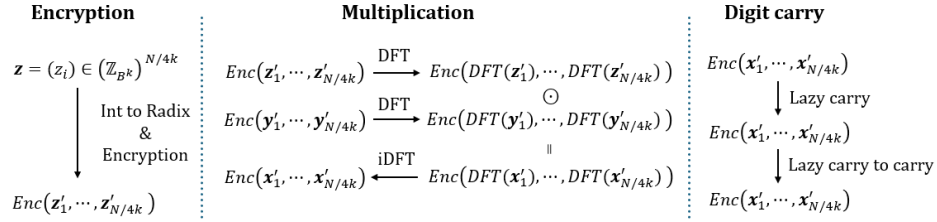


Fig. 1. This figure illustrates the overall process of decomposing an integer into its radix representation, encrypting it, and then performing polynomial arithmetic and handling digit carries.

across slots, making direct polynomial multiplication infeasible. To overcome this, we leverage homomorphic FFT transformations and perform multiplication in the FFT domain as element-wise operations. To prevent cyclic shift effects in FFT form, k zeros are padded. As a result, for $N/2$ CKKS slots, a total of $N/4k$ large integers can be stored.

After performing the FFT-based polynomial multiplication, we convert the result back to its unique digit representation. This process consists of a two-step algorithm, which we define as "lazy carry" (LC) and "lazy carry to carry" (LCtoC). The overall process of our scheme is depicted in Figure 1.

We now present the core building blocks of the radix-based CKKS scheme, with detailed explanations given in Sections 3 and 4.

Homomorphic lazy digit carry algorithm Polynomial operations produce valid integers when substituting $X = B$, but they do not yield a unique digit representation. The main issue is that magnitude of digits grow after repeated operations, and CKKS noise also grows proportionally to the message size during homomorphic multiplications. Without controlling magnitude growth, decryp-

tion cannot yield exact results. To address this, we introduce the LC algorithm that reduces magnitude of digits while preserving the represented integer value, following the lazy (or approximate) reduction philosophy of [6]. Specifically, we compute ciphertexts holding both remainder and quotient modulo B using the modular reduction method of [26], and then propagate the quotient to the next digit via CKKS rotation. Finally, since the last k slots represent multiples of B^k , multiplying by a masking vector to zero them out achieves B^k reduction. In short, our LC algorithm converts digits as:

$$z(X) = \sum_{0 \leq i < 2k} z_i \cdot X^i \implies z'(X) = [z_0]_B + \sum_{1 \leq i < k} ([z_i]_B + Q_B(z_{i-1}))X^i.$$

Homomorphic exact digit carry algorithm While the LC algorithm rapidly reduces the magnitude of digits, ensuring exact correctness may require up to k iterations in the worst case. Since homomorphic encryption leaks no information about the message, we must always assume this worst-case scenario. However, the digit magnitude remains bounded.

Once this bound is reduced below $2B - 1$, digit carry for base B becomes equivalent to binary carry: each digit propagates either 0 or 1 to the next position. Even when a carry is received from the previous digit, its value stays below $2B - 1$. Therefore, the carry behavior depends only on whether a digit propagates 1, which is analogous to binary behavior.

Based on this observation, we define the LCtoC procedure as follows:

1. Apply LC to reduce each digit's bound to $2B - 1$.
2. Use an interpolation polynomial to map each digit into $\{0, 1, 2\}$ according to its range.
3. With CKKS rotation, evaluate a two-variable interpolation function that takes the current and previous digit as inputs.

The function outputs whether carry propagation occurs: 0 = false, 1 = hold, 2 = true. Each digit then carries a propagation flag, which is used to perform the exact digit carry for base B .

Homomorphic comparison of large integers From basic number theory, for $c = a - b \bmod B^k$, if $a \geq b$ then $c = a - b$, otherwise $-B^k + c = a - b$. Thus, comparison reduces to checking the highest digit of c . To enable this, we subtract two integers and apply the digit carry algorithm to recover the highest digit. However, subtraction may produce negative digits, for which the standard digit carry algorithm is not directly applicable. We therefore adapt the algorithm to handle negative digits, enabling ciphertext-ciphertext comparison and conditional subtraction.

Homomorphic modular arithmetic With the above building blocks, we can homomorphically design and implement general-purpose reduction algorithms

such as Montgomery reduction. Traditionally, the Montgomery reduction algorithm involves reduction modulo $R = 2^r$, bit shifts by r , and conditional subtraction on the input T . In our scheme, by setting $R = B^k$, reduction can be implemented via multiplication with a masking vector, while bit-shifting is carried out using CKKS rotations. To accommodate intermediate values, however, the length parameter must be chosen as $2k$ rather than k . Additionally, for Generalized Mersenne Numbers (GMNs) such as elliptic curve primes, we design a homomorphic reduction algorithm called *base reduction*. This is a homomorphically-friendly reconstruction of the *folding method*.

1.3 Related Works

High-precision HE has been widely studied not only in CKKS but also in BFV/BGV and DM/CGGI. We review prior works by categorizing them according to the underlying HE scheme.

BFV/BGV For integer computations, the BGV and BFV schemes are typically the standard choice, as they naturally support arithmetic over $\mathbb{Z}_t$. However, achieving high-precision operations requires a large plaintext modulus t , which in turn increases noise proportionally and thus demands larger parameters. To address this, [12] proposed using a polynomial plaintext modulus of the form $t = X - b$. This design significantly reduces noise while enabling high-precision arithmetic. Nevertheless, bootstrapping remained an open problem for this construction and was not well understood until recently.

Subsequently, [19] introduced a generalization between BFV and CLPX, called generalized BFV (GBFV). The key idea is that choosing $t(X) = X^k - b$ ensures that the GBFV plaintext space $\mathcal{R}_t$ becomes a subspace of the BFV plaintext space $\mathcal{R}_p$, where $p = \Phi_m(b^{m/k})$. This allows GBFV to support computations under various modulus and to perform bootstrapping by converting back to BFV. However, GBFV faces several constraints. (1) The modulus p depends on cyclotomic polynomials. (2) Since bootstrapping relies on BFV conversion, it remains infeasible for very large p . To overcome this, [23] showed that GBFV bootstrapping for large p can be achieved by adapting the approach of [27]. Yet, the iterative bootstrapping process and modulus consumption make the method impractical in many settings.

Finally, in the (G)BFV family, the number of slots depends on t , since the degree of parallelism is determined by how t splits the underlying ring $\mathcal{R}$.

DM/CGGI Research on the DM/CGGI scheme has explored several directions: mixed-radix approaches [15], CRT-based approaches [10,11], and hybrid methods combining both [5]. In all cases, a large integer is divided into smaller chunks, and each chunk is encrypted separately. While this enables flexible computation, performing even basic arithmetic across multiple ciphertexts requires numerous instances of functional bootstrapping. Although DM/CGGI provides relatively fast bootstrapping, the cumulative cost remains significant.

Discrete CKKS In contrast, the CKKS scheme performs integer-like computations by embedding them into arithmetic over $\mathbb{C}$. Unlike BFV/BGV or DM/CGGI, discrete CKKS can store large values directly, but it lacks natural support for modulus t arithmetic.

[6] homomorphically evaluates the fast base conversion of [21] over the RNS representation $\mathbb{Z}_p = \prod \mathbb{Z}_{p_i}$. This enables reductions for $r < \sqrt{p}$. This procedure is referred to as *first-layer* base conversion. Arithmetic operations on each element in $\mathbb{Z}_{p_i}$ are carried out in a slot-wise manner, and the modular reduction $[\cdot]_{p_i}$ is performed using the technique proposed in [26]. However, due to the size constraints of p_i that can be handled by modular reduction, the first-layer RNS is limited in the size of modulus it can support.

To overcome this limitation, the authors propose a *nested* RNS, in which each coprime of the second-layer RNS is reduced within the first layer. Specifically, for $\mathbb{Z}_r = \prod \mathbb{Z}_{r_i}$, arithmetic operations $[\cdot]_{r_i}$ are performed in each first-layer component. For $s < \sqrt{r}$, another base conversion is homomorphically evaluated, which is referred to as the *second-layer* base conversion. However, this reduction does not guarantee exactness with respect to the target modulus. Its applicability is therefore limited to addition and multiplication, while operations such as comparison remain difficult due to structural constraints of RNS.

In addition to [6], several works on high-precision CKKS adopt radix-based approaches [24,25]. The method of [24] encrypts each digit of a B^k representation into a separate ciphertext. While this provides excellent parallelism, it incurs significant memory and latency overhead as the bit-width grows, since k ciphertexts and $O(k)$ bootstraps are required for digit carries. By contrast, [25] leverages CKKS encoding for polynomial operations within a single ciphertext and proposes a reduction algorithm using $O(\log k)$ bootstraps to control digit growth. Restoring a unique digit representation, however, still requires an exact digit-carry procedure with $O(k)$ bootstrapping complexity.

Beyond arithmetic operations, functional bootstrapping for high-precision LUT evaluation has also been explored [2,17]. These studies improve precision by slightly increasing the parameter N or by homomorphically decomposing the message into digits. In practice, reported LUT evaluations reach at most 20-bit precision. In contrast, our work targets higher precision for specific operations, in particular multiplication and comparison. Our technique requires restoration to a unique digit representation (a prerequisite for LUT evaluation), but we do not address arbitrary LUT functions.

2 Preliminaries

Let N be an integer that is a power-of-two. Let $Q > 0$ be an integer, $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$, and $\mathcal{R}_Q = \mathcal{R}/Q\mathcal{R}$. The vectors are denoted in bold lower case (e.g. $\mathbf{v}, \mathbf{w}$). Given an integer z , $[z]_B$ is the remainder modulo B and a regular representation of $\mathbb{Z}_B$. Furthermore, $Q_B(z)$ is the quotient of z satisfying $z = Q_B(z) \cdot B + [z]_B$. For a vector $\mathbf{v}$, we define the infinity norm as $\|\mathbf{v}\|_\infty = \max_i |v_i|$.

For a brief overview of the CKKS scheme and the notation used in this paper, we refer the reader to A.

2.1 Discrete CKKS

Discrete CKKS is a kind of modification that breaks away from the conventional concept of handling complex vectors and instead handles discrete data such as integers. Discrete CKKS differs from BFV/BGV and DM/CGGI, which were considered standard for existing integer data types. First, CKKS handles integers as a subspace of the complex number field rather than a finite field. This difference means that, unlike BFV/BGV, which has native reduction, there is no overflow phenomenon, and both real and integer data types can be handled simultaneously. The second difference comes from the CKKS philosophy of not separating errors. Since CKKS does not have a native algorithm to reduce errors, errors must be reduced as needed when processing discrete data. This process is called error cleaning.

Discrete Bootstrapping State-of-the-art discrete bootstrapping techniques [4,2,17] support functional bootstrapping on integers while simultaneously performing error cleaning. This paper addresses the simplest form of discrete bootstrapping. In what follows, we briefly describe each subroutine of the approach used in this work.

1. **Slots-to-Coefficients (StC)** : Given a ciphertext $\text{ct} = \text{Enc} \circ \text{Ecd}(\mathbf{z}) \in \mathcal{R}_q^2$ encrypting a message $\mathbf{z} \in \mathbb{Z}_t$, the output ciphertext $\text{ct}' = \text{Enc}(q_0/t \cdot \mathbf{z}(X)) \in \mathcal{R}_{q_0}^2$ where $\mathbf{z}(X)$ is a polynomial in $\mathcal{R}$ whose coefficients corresponds to the entries of $\mathbf{z}$. This means moving the slot to the coefficient.
2. **Modulus Raising (ModRaise)** : Given a ciphertext $\text{ct} = \text{Enc}(q_0/t \cdot \mathbf{z}(X)) \in \mathcal{R}_{q_0}^2$, the output ciphertext $\text{ct}' = \text{Enc}(q_0/t \cdot \mathbf{z}(X) + q_0 \cdot I(X)) \in \mathcal{R}_Q^2$ where $I(X)$ is a polynomial in $\mathcal{R}$ is a small integer polynoial and $Q > q_0$.
3. **Coefficients-to-Slots (CtS)** : Given a ciphertext $\text{ct} = \text{Enc}(q_0/t \cdot \mathbf{z}(X) + q_0 \cdot I(X)) = \text{Enc}(q_0/t \cdot (\mathbf{z}(X) + t \cdot I(X))) \in \mathcal{R}_Q^2$, the output ciphertext $\text{ct}' = \text{Enc} \circ \text{Ecd}(\mathbf{z} + t \cdot \mathbf{I})$. This means moving the coefficient to the slot.
4. **Homomorphic Exponentiation (EvalExp)** : Given a ciphertext $\text{ct} = \text{Enc} \circ \text{Ecd}(\mathbf{z} + t \cdot \mathbf{I})$, the output ciphertext $\text{ct}' = \text{Enc} \circ \text{Ecd}(\exp(2\pi i \cdot \mathbf{z}/t))$ by homomorphically evaluating a complex exponential $x \rightarrow \exp(2\pi i \cdot x/t)$.
5. **Homomorphic Look-up Table (LUT_f)** : Given a ciphertext $\text{ct} = \text{Enc} \circ \text{Ecd}(\exp(2\pi i \cdot \mathbf{z}/t))$, the output ciphertext $\text{ct}' = \text{Enc} \circ \text{Ecd}(f(\mathbf{z}))$ by homomorphically evaluating a k th order hermite interpolation $\exp(2\pi i \cdot x/t) \rightarrow f(x)$. Since Hermite interpolation is used, error cleaning is performed in this subroutine.

We describe the discrete bootstrapping procedure in Algorithm 1 and name the algorithm diBtp_f .

Modular Reduction As mentioned earlier, unlike BFV/BGV, discrete CKKS does not natively support reduction functions. However, recent works [26,2] have demonstrated that such a function can be embedded directly into the bootstrapping procedure.

The core idea is to adjust the scaling factor to q_0/t during the StC (slot-to-coefficient) conversion. Subsequently, when the reduction is performed with respect to the base modulus q_0 , the CKKS ciphertext can be interpreted as a coefficient-packed BFV ciphertext with a plaintext modulus t over $\mathcal{R}_{q_0}$. In this representation, any components of the coefficients that are multiples of t are naturally reduced modulo q_0 . As a result, each coefficient effectively represents a value modulo t . After the remaining bootstrapping subroutines are executed, the final ciphertext contains values in the slots that are already reduced for t .

This modular reduction technique was developed almost concurrently by two research groups. The studies [26] and [2] named their methods IntMod_t and HomFloor_t , respectively. While the two approaches are nearly identical, they differ slightly in their output: IntMod_t returns $[x]_t$, whereas HomFloor_t returns $x - [x]_t$. In this paper, we follow the terminology of [26] and refer to this procedure as IntMod_t . In essence, this procedure is identical to the diBtp_f , with the key exception of the scaling adjustment. The full procedure is summarized in Algorithm 2.

Algorithm 1 diBtp_f

Setting: $\Delta_0 = q_0/t$

Input: $\text{ct} = \text{Enc} \circ \text{Ecd}(\mathbf{z}) \in \mathcal{R}_q^2$, $\mathbf{z} \in \mathbb{Z}_t^{N/2}$

Output: $\text{ct}_{\text{out}} = \text{Enc} \circ \text{Ecd}(\mathbf{z}) \in \mathcal{R}_Q^2$

- 1: $\text{ct}_1 \leftarrow \text{CtS} \circ \text{ModRaise} \circ \text{StC}(\text{ct})$
 - 2: $\text{ct}_2 \leftarrow \text{EvalExp}(\text{ct}_1)$
 - 3: $\text{ct}_{\text{out}} \leftarrow \text{LUT}_f(\text{ct}_2)$
 - 4: **return** ct_{out}
-

Algorithm 2 IntMod_t

Setting: $\Delta_0 = q_0/t$

Input: $\text{ct} = \text{Enc} \circ \text{Ecd}(\mathbf{z}) \in \mathcal{R}_q^2$, $\mathbf{z} \in \mathbb{R}^{N/2}$

Output: $\text{ct}_{\text{out}} = \text{Enc} \circ \text{Ecd}(\mathbf{z}) \in \mathcal{R}_Q^2$

- 1: $\text{ct}_1 \leftarrow \text{StC}(\text{ct})$
 - 2: $\text{ct}_2 \leftarrow [\text{ct}_1]_{q_0}$
 - 3: $\text{ct}_3 \leftarrow \text{CtS} \circ \text{ModRaise}(\text{ct}_2)$
 - 4: $\text{ct}_4 \leftarrow \text{EvalExp}(\text{ct}_3)$
 - 5: $\text{ct}_{\text{out}} \leftarrow \text{LUT}_f(\text{ct}_4)$
 - 6: **return** ct_{out}
-

3 The Radix-based CKKS scheme

In this section, we describe our radix-based CKKS scheme for B^k arithmetic. We begin by loosely defining radix representations of integers.

Definition 1. Let z be an integer in the interval $[0, B^k)$. We say that a vector $\mathbf{z} \in \mathbb{Z}^n$ is the radix representation of z with respect to B if it satisfies

$$z = \sum_{0 \leq i < n} z_i \cdot B^i \pmod{B^k}.$$

In particular, if every component of $\mathbf{z}$ is non-negative and less than B , we call $\mathbf{z}$ the unique radix representation of z .

The radix representation $\mathbf{z} = (z_i)_{0 \leq i < n}$ of z is often expressed as a polynomial

$$z(X) = \sum_{0 \leq i < k} z_i X^i.$$

Note that the vector representation may include trailing zeros, whereas the polynomial representation is always of degree at most $k - 1$.

Encoding For positive integers B and k , we define the mapping from $z \in \mathbb{Z}_{B^k}$ to its unique radix representation, and its inverse:

$$\begin{aligned} D_{B,k} : \mathbb{Z}_{B^k} &\rightarrow \mathbb{Z}^{2k}, \quad z \mapsto \mathbf{z} = (z_0, z_1, \dots, z_{k-1}, 0, \dots, 0), \\ D_{B,k}^{-1} : \mathbb{Z}^{2k} &\rightarrow \mathbb{Z}_{B^k}, \quad \mathbf{z} \mapsto z = \sum_{0 \leq i < 2k} z_i \cdot B^i. \end{aligned}$$

These maps extend naturally to multiple inputs:

$$\begin{aligned} D'_{B,k} : \mathbb{Z}_{B^k}^{N/4k} &\rightarrow \mathbb{Z}^{N/2}, \quad (z^{(i)})_{0 \leq i < N/4k} \mapsto (D_{B,k}(z^{(i)}))_{0 \leq i < N/4k}, \\ D'^{-1}_{B,k} : \mathbb{Z}^{N/2} &\rightarrow \mathbb{Z}_{B^k}^{N/4k}, \quad (\mathbf{z}'_i = (z_{i \cdot 2k + j})_{0 \leq j < 2k})_{0 \leq i < N/4k} \mapsto (D_{B,k}^{-1}(\mathbf{z}'_i))_{0 \leq i < N/4k} \end{aligned}$$

where $k \mid N/2$. For clarity, we also extend the definition of radix representation: given $\mathbf{z} \in \mathbb{Z}_{B^k}^{N/4k}$, its radix representation $\mathbf{z}' \in \mathbb{Z}^{N/2}$ is

$$z_i = \sum_{j=0}^{2k-1} z_{i \cdot 2k + j} \cdot B^j \pmod{B^k}, \quad 0 \leq i < N/4k.$$

To reduce the number of CKKS rotations required in linear transformations, we apply a special permutation to the slot vector prior to encoding. Let $\pi_k : \{0, 1, \dots, N/2 - 1\} \rightarrow \{0, 1, \dots, N/2 - 1\}$ be such a permutation. For $\mathbf{z} = (z_0, z_1, \dots, z_{N/2-1})$, we define

$$\pi(\mathbf{z}) = (z_{\pi(i \cdot 2k + j)}) = (z_{j \cdot N/4k + i}), \quad 0 \leq i < N/4k, \quad 0 \leq j < 2k.$$

When describing our algorithms, we often use the j -shift rotation ($0 \leq j < 2k$). Importantly, under the permutation π , a j -shift rotation is equivalent to a $j \cdot N/(4k)$ -shift rotation:

$$\rho_{j \cdot N/4k} \circ \pi(\mathbf{z}) = \pi \circ \rho_j(\mathbf{z}), \quad 0 \leq j < N/4k.$$

Thus, for simplicity, we omit the permutation when discussing rotations.

Using the extended functions and permutation, the encoding and decoding functions are defined as

$$\text{REcd} : \mathbb{Z}_{B^k}^{N/4k} \rightarrow \mathcal{R}, \quad \mathbf{z} \mapsto \text{Ecd} \circ \pi \circ D'_{B,k}(\mathbf{z}),$$

$$\text{RDcd} : \mathcal{R} \rightarrow \mathbb{Z}_{B^k}^{N/4k}, \quad m(X) \mapsto D'^{-1}_{B,k} \circ \pi^{-1} \circ \text{Dcd}(m).$$

Addition / Multiplication Addition in our scheme directly follows CKKS addition. Multiplication, however, requires polynomial multiplication rather than slot-wise multiplication. To achieve this, we use discrete Fourier transformations¹

$$\text{PolyMult}(\text{ct}_1, \text{ct}_2) = \text{HomDFT}_{2k}^{-1} \left(\text{HomDFT}_{2k}^*(\text{ct}_1) \odot \text{HomDFT}_{2k}^*(\text{ct}_2) \right).$$

Here, HomDFT_{2k} denotes the homomorphic linear transformation defined by the matrix $A = (\zeta^{ij})$, where $\zeta = \exp(\pi i/k)$. We apply this transformation to each of the $N/4k$ vectors. The asterisk $*$ notation represents the normalized DFT.

The linear transformation is performed via the baby-step giant-step algorithm, requiring $2\sqrt{2k}$ key switching operations for each transformation. While the Cooley-Tukey algorithm can improve the efficiency of PolyMult , it has the drawback of producing a bit-reversed output. Since the original output order must be maintained for the subsequent digit carry algorithm, we do not use the Cooley-Tukey algorithm.

After PolyMult , if reduction is required, we apply a mask that zeroes out the last k slots. In practice, this masking is embedded directly into the inverse DFT matrix A^{-1} . If reduction is not applied, $2k$ zero-padded slots must be reserved for further multiplications. For fully packed ciphertexts, masking is used to separate the two ciphertexts containing $N/8k$ integers.

Digit carry The cornerstone of our scheme is a two-step digit carry algorithm, comprising the LC procedure and an exact carry procedure, **LCtoC**.

$$\begin{aligned} \text{LC}(\text{ct}) : & \text{ Given } \text{ct} = \text{Enc} \circ \text{Ecd}(\mathbf{z}), \text{ where } \mathbf{z} \text{ is a radix representation of } \mathbf{y}, \\ & \text{ output } \text{ct}' = \text{Enc} \circ \text{Ecd}(\mathbf{z}') \text{ (same } \mathbf{y} \text{) with } \|\mathbf{z}'\|_\infty < \|\mathbf{z}\|_\infty. \end{aligned}$$

¹ For clarity, we make a distinction in terminology. The DFT transformation underlying the CKKS encoding is referred to as the “encoding matrix,” whereas the homomorphic transformation for polynomial multiplication is denoted as HomDFT or simply DFT.

$\text{LCtoC}(\text{ct})$: Given a ciphertext $\text{ct} = \text{Enc} \circ \text{Ecd}(\mathbf{z})$, where $\|\mathbf{z}\|_\infty < 2B - 1$
and $\mathbf{z}$ is a radix representation of $\mathbf{y}$, output $\text{ct}' = \text{Enc} \circ \text{REcd}(\mathbf{y})$.

The first step, LC, is a magnitude reduction algorithm (performed via bootstrapping) that reduces the magnitude of digits after an arithmetic operation. Its primary purpose is to control the message size, thereby mitigating the growth of multiplication noise inherent in CKKS. The second step, LCtoC, takes this magnitude-controlled ciphertext as input and restores it to a precise, unique radix representation. Concretely, for a bit-width k , LCtoC performs one bootstrapping and then consumes an additional multiplicative depth of $\log k + 1$. While the exact number of bootstrappings can vary slightly with the CKKS parameters, it remains near-constant overall.

Theorem 1. *For two ciphertexts encrypting large integers with a unique radix representation, the number of bootstrappings required to restore that unique representation after an addition or multiplication is $O(1)$ and $O(\log k)$, respectively. In other words, this is the number of bootstrappings used in the full digit carry operation, $\text{LCtoC} \circ \text{LC}^t(\cdot)$.*

Theorem 1 proves that our complete digit carry algorithm requires a number of bootstraps of $O(\log k)$ with respect to the bit-width k . The descriptions of the LC and LCtoC algorithms will be covered in Sections 3.1 and 3.3, while the proof of Theorem 1 will be discussed in Section 3.4

3.1 Lazy Carry Algorithm

In the homomorphic RNS of [6], for integers p, r with $r^2 < p$, the operation $[\cdot]_r$ over $\mathbb{Z}_p$ is performed lazily. This involves allowing a small error re to satisfy the condition $([\cdot]_r + re)^2 < p$, which ensures the correctness of subsequent operations.

Our LC algorithm is inspired by this philosophy of lazy reduction [6]. Given $a, b \in \mathbb{Z}_{B^k}$, rather than performing exact digit-carrying after multiplication, we aim to reduce the result more efficiently into the form

$$[a \cdot b]_{B^k} + B^k e,$$

for some small e .

Let $z(X)$ denote the product of two polynomials of degree at most $k - 1$, each expressed in its unique radix representation. Then $\deg z(X) \leq 2k - 2$. A key advantage of the radix-based approach is that higher-indexed digits can store larger values. Thus, the lazy reduction $[\cdot]_{B^k}$ is easily performed by masking the last k slots with zeros. However, it is important to note that in CKKS, multiplication error grows proportionally to the message size. Even if masking reduces the decrypted value modulo B^k , the message size itself is not reduced. Indeed, an upper bound on $z(X)$ is kB^2 . For example, with $B = 2^4$ and $k = 16$

(corresponding to a 64-bit multiplication), the message size may grow up to 12 bits.

The LC algorithm reduces the magnitude of each digit by homomorphically evaluating the following equation:

$$z(X) = \sum_{0 \leq i < 2k} z_i \cdot X^i \longrightarrow [z_0]_B + \sum_{1 \leq i < k} ([z_i]_B + Q_B(z_{i-1})) \cdot X^i. \quad (1)$$

Theorem 2 (Correctness of LC). *Let ct be a ciphertext of a message $\mathbf{z} + \boldsymbol{\varepsilon} \in \mathbb{Z}^{N/2}$, where $\mathbf{z}$ is the radix representation of $\mathbf{y}$ and satisfies $\|\mathbf{z}\|_\infty < U_{\mathbf{z}}$. Let $\mathbf{v} = (v_i)_{0 \leq i < N/2}$ be the masking vector described in Algorithm 3. If $\|\boldsymbol{\varepsilon}\|_\infty < \varepsilon_b$, where ε_b is the theoretical upper bound on the error-cleaning capability of discrete bootstrapping. Then $\text{ct}' = \text{LC}(\text{ct})$ is a ciphertext of a message $\mathbf{z}'$, where $\mathbf{z}'$ is the radix representation of $\mathbf{y}$ and satisfies*

$$\|\mathbf{z}'\|_\infty < U_{\mathbf{z}'} = O\left(\frac{U_{\mathbf{z}}}{B}\right) + B.$$

Algorithm 3 LC

Setting: $\Delta_0 = q_0/B$

Input: $\text{ct} = \text{Enc} \circ \text{Ecd}(\mathbf{z}) \in \mathcal{R}_{q_\ell}^2$, where $\mathbf{z}$ is the radix-representation of $\mathbf{y}$ with $\|\mathbf{z}\|_\infty < U_{\mathbf{z}}$, and masking vector $\mathbf{v}$ that sets the last k slots to 0.

Output: $\text{ct}_{\text{out}} = \text{End} \circ \text{Ecd}(\mathbf{z}') \in \mathcal{R}_{q_{\ell-1}}^2$, where $\mathbf{z}'$ is the radix-representation of $\mathbf{y}$ with $\|\mathbf{z}'\|_\infty < Q_B(U_{\mathbf{z}}) + B$

- 1: $\text{ct}_{\text{mod}} \leftarrow \text{IntMod}_B(\text{ct})$
 - 2: $\text{ct}_Q \leftarrow (\text{ct} - \text{ct}_{\text{mod}})/B$
 - 3: $\text{ct}_{\text{out}} \leftarrow \text{ct}_{\text{mod}} + \rho_{-1}(\text{ct}_Q) \odot \mathbf{v}$
 - 4: **return** ct_{out}
-

Theorem 2 addresses the correctness of the LC algorithm, and Algorithm 3 presents its pseudocode. The proof provided in Appendix B.1

3.2 Exact Digit Carry Algorithm

In this section, we explain the LCtoC algorithm, which performs an exact digit carry. This algorithm operates under the assumption that each digit is bounded by $2B-1$. This precondition can be met by repeatedly applying the LC algorithm, and an analysis of the required number of iterations is provided in Section 3.3. Under the assumption discussed earlier, each digit can propagate at most 1 carry to the next, and since the following digit is at most $2B-2$, all carries are binary (either 0 or 1). Therefore, within this bounded digit range, the carry behavior is exactly the same as in binary carry.

To facilitate the explanation of our algorithm, it is necessary to clearly describe the behavior of a binary-like carry, even if it may seem obvious. Given a

carry operation on a digit vector $\mathbf{b} = (b_i)_{0 \leq i < k} \in \{0, 1, 2\}^k$, let us examine the condition on the sub-vector $\mathbf{b}[0 : \ell - 1] = (b_i)_{0 \leq i < \ell}$ that causes a carry to be propagated to the ℓ th position for $0 \leq \ell < k$.

- If the $\ell - 1$ th digit is 0, no carry is propagated to the ℓ th position. Even with a carry-in from the previous digit, the value becomes 1, which does not generate a further carry.
- If the $\ell - 1$ th digit is 2, a carry of 1 is propagated to the ℓ th position.
- If the $\ell - 1$ th digit is 1, propagation depends on the $\ell - 2$ th digit. If the $\ell - 2$ th digit is 0, no carry occurs. If it is 2, a carry occurs. If it is 1, the propagation is determined by the $\ell - 3$ th digit. In other words, this is determined recursively.

Therefore, if the $(\ell - 1)$ -th position of the digit vector $\mathbf{b}$ propagates a 1 to the ℓ -th position, the tail vector of the sub-vector $\mathbf{b}[0, \ell - 1]$ must have the regular expression $\{2, \{1\}^*\}$. A carry propagation means subtracting 2 from the current digit and adding 1 to the next. Within the given digit range, the behavior of a carry for base B is identical to that of a carry for binary. Therefore, after computing a vector of 1s or 0s for the propagation status, the carry can be performed by subtracting B instead of 2.

In summary, when executing the carry algorithm for base B , it is unnecessary to consider the entire range. The process simplifies to mapping each value to 0, 1, or 2 based on its interval, and then determining the carry status (i.e., the shape of the tail) for each position. Based on these results, we design the homomorphic carry algorithm LCtoC. Its overall procedure is given in Algorithm 4.

Algorithm 4 LCtoC

Input: $\mathbf{ct} = \text{Enc} \circ \text{Ecd}(\mathbf{z}) \in \mathcal{R}_q^2$, where $\mathbf{z}$ is the radix-representation of $\mathbf{y}$ with $\|\mathbf{z}\|_\infty < 2B - 1$

Output: $\mathbf{ct}_{\text{out}} = \text{Enc} \circ \text{REcd}(\mathbf{y}) \in \mathcal{R}_Q^2$

- 1: $\mathbf{ct}_{\text{symbol}} \leftarrow \text{diBtp}_\phi(\mathbf{ct})$
 - 2: **for** $i = 0$ to $\log(k)$ **do**
 - 3: $\mathbf{ct}_{\text{tmp}} \leftarrow \rho_{-2^i}(\mathbf{ct}_{\text{symbol}})$
 - 4: $\mathbf{ct}_{\text{symbol}} \leftarrow \text{Evaluate } f(\mathbf{ct}_{\text{tmp}}, \mathbf{ct}_{\text{symbol}})$
 - 5: **end for**
 - 6: $\mathbf{ct}_u \leftarrow \text{Evaluate } \tau(\mathbf{ct}_{\text{symbol}})$
 - 7: $\mathbf{ct}_{\text{out}} \leftarrow \mathbf{ct} - B \cdot \mathbf{ct}_u + \rho_{-1}(\mathbf{ct}_u)$
 - 8: **return** $\mathbf{ct}_{\text{out}}$
-

The first step is to perform bootstrapping in order to evaluate the look-up table ϕ defined below.

$$\phi : [0, 2B - 1) \cap \mathbb{Z} \rightarrow \{0, 1, 2\} : \phi(z) = \begin{cases} 0 & \text{if } 0 \leq z < B - 1, \\ 1 & \text{if } z = B - 1, \\ 2 & \text{if } B \leq z < 2B - 1. \end{cases}$$

For correctness, the scaling factor must be set as $\Delta_0 = q_0/(2B - 1)$.

Next, we define a two-variable interpolation function f that takes as input the current digit y and the previous digit x and returns the carry status:

$$f(x, y) = \begin{cases} 0 & \text{if } y = 0, \\ x & \text{if } y = 1, \\ 2 & \text{if } y = 2. \end{cases}$$

This function can be expressed as $f(x, y) = 0 \cdot L_0(y) + x \cdot L_1(y) + 2 \cdot L_2(y) = y^2 - y + xy(2 - y)$, where $L_k(y)$ equals 1 when $y = k$ and 0 otherwise. The evaluation of this function requires depth 2 and one rotation. Since the last k digits of the previous integer are set to zero (which does not affect carry propagation), we can compute the carry status for all positions by evaluating f recursively. One might expect that computing the carry status would require k evaluations of f along with k rotations, corresponding to a sequential process. However, this can actually be achieved in only $\log k$ evaluations and $\log k$ rotations. To explain simply, this optimization is analogous to calculating the carry status by progressively increasing the base. The first iteration computes the carry for all individual digits (i.e., in base B). The second iteration groups adjacent pairs to compute the carry status for 2-digit blocks (effectively treating them as digits in base B^2). This process is repeated, doubling the base (e.g., to base B^4 , then base B^8) at each step. A formal mathematical proof for this approach is provided in Lemma 2 of Appendix B.2.

Next, we evaluate the function τ , which maps $\{0, 1, 2\}$ to $\{0, 0, 1\}$:

$$\tau(x) = \frac{1}{2}x(x - 1),$$

which also requires depth 2.

Finally, Let the ciphertext obtained at this stage be denoted ct_u . This ciphertext encodes the carry behavior of the input: if the ℓ th digit of ct_u is 1, then the ℓ th digit is reduced modulo B and a carry of 1 is propagated to the $(\ell + 1)$ th digit; if it is 0, no propagation occurs.

Accordingly, subtracting the input ciphertext from ct_u multiplied by B achieves reduction by B , and then a single right rotation of ct_u followed by addition propagates 1.

The evaluation process of LCtoC is illustrated step-by-step in Figure 2. The proof of correctness for the algorithm is given in Theorem 3.

Change the Symbol The LCtoC algorithm requires a level consumption of $2\log(k) + 2$, which is prohibitively high given the level budget available for discrete bootstrapping. To address this, we observe that the values $\{0, 1, 2\}$ are purely symbolic, and thus we can remap these symbols to complex numbers in order to reduce the level consumption of the LCtoC algorithm.

First, our function can be written as follows:

$$f(x, y) = \begin{cases} x & \text{if } y = 1, \\ y & \text{if } y \neq 1 \end{cases}$$

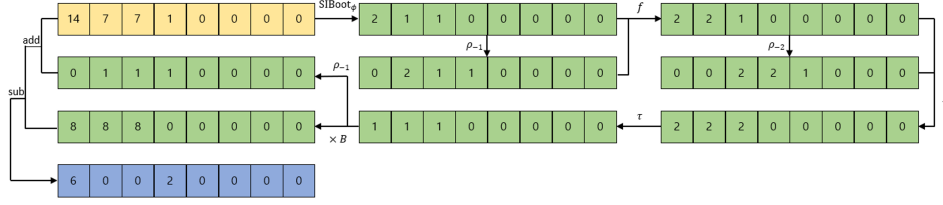


Fig. 2. The LCtoC algorithm for $(B, k) = (8, 4)$. Yellow blocks represent the input ciphertext after lazy multiplication, and blue blocks represent the output ciphertext after exact digit carry.

With this formulation, $f(x, y)$ can be interpolated as

$$f(x, y) = xL_1(y) + y(1 - L_1(y)).$$

Instead of constructing $L_1(y)$ using Lagrange interpolation, we propose a different encoding method that utilizes conjugation operations. If we change the symbol encoding from $\{0, 1, 2\}$ to $\{0, \frac{1}{2}, i\}$, then $L_1(y) = 2 \cdot \text{Re}(y) = y + \bar{y}$ can serve as the interpolation function. That is, $L_1(\frac{1}{2}) = 1$ and $L_1(0) = L_1(i) = 0$. Note that $L_1(y)$ requires no multiplicative depth. In addition, we can observe that the function τ , mapping $x = i$ to 1, and $x = 0$ and $x = \frac{1}{2}$ to 0, extracts the imaginary part while discarding the real part. Combining these observations, f and τ can be constructed as follows:

$$f(x, y) = xL_1(y) + y(1 - L_1(y)) = y + (y + \bar{y})(x - y)$$

$$\tau(x) = -i \cdot \text{Im}(x) = \frac{i}{2}(\bar{x} - x).$$

Both of these functions use depth 1, and therefore, the LCtoC algorithm on the new symbol encoding requires $\log(k) + 1$ depth in total.

Finally, we define two multiplication algorithms: - **LazyMult**, which reduces magnitude of digits through repeated applications of LC, and - **ExactMult**, which performs exact carry propagation using LCtoC. The detailed procedures for LazyMult and ExactMult are presented in algorithm 5.

3.3 Noise Analysis

In this section, we analyze the noise growth of the ExactMult algorithm. This analysis is crucial, as controlling noise growth directly determines the correctness of decryption.

As stated in Theorem 2, for LC to operate correctly, the error magnitude must remain below ε_b . Although the LC procedure inherently reduces part of the accumulated error, if the noise introduced by PolyMult exceeds this bound, LazyMult can no longer guarantee correct decryption.

Algorithm 5 LazyMult, ExactMult

Setting: $t = t(B, k)$ is the number of LC operations

Input: $\text{ct}_i = \text{Enc} \circ \text{Ecd}(z_i) \in \mathcal{R}_q^2$, where z_i is the radix-representation of y_i , for $i = 1, 2$

Output: $\text{ct}_{\text{out}} = \text{Enc} \circ \text{Ecd}(z') \in \mathcal{R}_Q^2$, where z' is the radix-representation of $[y_0 \odot y_1]_{B^k}$
with $\|z'\|_\infty < 2B - 1$ or $\text{ct}_{\text{out}} = \text{Enc} \circ \text{REcd}([y_1 \odot y_2]_{B^k}) \in \mathcal{R}_Q^2$

1: $\text{ct}^{(0)} \leftarrow \text{PolyMult}(\text{ct}_1, \text{ct}_2)$

2: **for** $i = 0$ to t **do**

3: $\text{ct}^{(i+1)} \leftarrow \text{LC}(\text{ct}^{(i)})$

4: **end for**

5: $\text{ct}_{\text{out}} \leftarrow \text{LCtoC}(\text{ct}^{(t)})$

▷ LazyMult procedure returns $\text{ct}^{(t)}$

6: **return** ct_{out}

Since the noise in PolyMult is dominated by the HomDFT, controlling the noise of this transformation is the key. There are two main approaches: one is to enlarge the plaintext scaling factor used in the HomDFT, and the other is to decompose the HomDFT so as to reduce the numerical precision required for the complex transform matrix. Both methods can be viewed as ways to enhance the precision of the HomDFT. Indeed, as long as adequate precision is guaranteed, the noise generated in PolyMult does not pose a significant risk.

Unlike PolyMult, the error arising from LCtoC directly affects the efficiency and correctness of the algorithm. The proof is provided in Appendix B.2.

Theorem 3 (Correctness of LCtoC). *Let ct be a ciphertext encrypting a message $z + \varepsilon$, where $z \in \mathbb{Z}^{N/2}$ denotes the radix representation of y with $\|z\|_\infty < 2B - 1$. Suppose that $\|\varepsilon\|_\infty < \varepsilon_b$, where ε_b is the theoretical upper bound on the error-cleaning capability of discrete bootstrapping. Let $\text{ct}' = \text{Enc} \circ \text{Ecd}(z') = \text{diBtp}_f(\text{ct})$, and denote the ℓ_∞ -norm of resulting error by ε' . If*

$$5^{\log(k)/2} \cdot B \cdot \varepsilon' < \varepsilon_b,$$

then LCtoC is correct. Otherwise, the correctness of LCtoC can still be ensured by applying a small number of additional error-cleaning procedures.

This bound completes the error growth analysis of LCtoC. For a detailed discussion on additional cleaning, we refer to Section 3.5.

3.4 Efficiency Analysis

We now analyze the efficiency of our scheme. We have not yet explicitly discussed the digit carry procedure for addition. If two ciphertexts are stored with unique radix representations, the digits of their sum are bounded by $2B - 1$. Therefore, applying the LCtoC algorithm just once is sufficient to recover the unique representations.

Our primary focus is the efficiency of multiplication, the ExactMult algorithm. The following theorem specifies the number of LC iterations required to reduce the digit size to less than $2B - 1$ following a PolyMult operation.

Lemma 1. *Let B and k be positive integers. Consider a vector $\mathbf{z} \in \mathbb{Z}^{2k}$ as a base- B radix representation. If the digits of $\mathbf{z}$ satisfy the bound $\|\mathbf{z}\|_\infty < B^t$, then applying Equation (1) t times will reduce the digit bound to less than $2B - 1$. Therefore, after one multiplication, the number of bootstrappings required to reduce the digit bound to $2B - 1$ is $O(\log k) = \log_B(kB^2)$.*

Theorem 1 follows from Lemma 1, whose proof is provided in Appendix B.3. For addition, since the digit bound is $2B - 1$, the unique digit representation can be restored with a single call to **LCtoC**. For multiplication, the restoration requires $O(\log k)$ iterations of **LC** followed by one call to **LCtoC**.

We now turn to the efficiency analysis, with a particular focus on the scheme parameters (B, k) . First, we fix a modulus $p = B^k$. Choosing a larger B results in a smaller k , which increases throughput and reduces the number of iterations for the **LC** procedure. However, this choice presents two main drawbacks. First, a larger B increases the degree of the interpolation polynomial for the **LUT** operation, elevating its complexity and latency during discrete bootstrapping. Second, since error growth in the **LCtoC** procedure is linear with respect to B , a larger base may necessitate more bootstrapping operations for error cleaning.

These trade-offs are further complicated by the fact that the size of the base B is inherently constrained. Consequently, processing a large modulus p (e.g., 2^{2048}) requires a large value for k . This poses a significant challenge for the **LCtoC** procedure, which needs a circuit depth of $\log(k)$, and thus requires bootstrapping to secure the level budget. In this scenario, it is desirable to use **diBtp** to perform error cleaning and secure the level budget simultaneously. However, because the need for error cleaning and the depletion of the level budget do not coincide, careful parameter tuning is required to execute the procedure with minimal bootstrapping.

3.5 SymbolCleaning : Error Cleaning for **LCtoC**

As discussed in Theorem 3, an error cleaning procedure is essential for managing the error within the **LCtoC** algorithm. What distinguishes this cleaning step from conventional methods is that we only need to interpolate three complex points: $\{0, 1/2, i\}$. We leverage this specific condition to apply a slight optimization to the error cleaning process.

Cleaning with Bootstrapping Note that the **CtS** returns two ciphertexts, corresponding to the real and imaginary parts of the slots. That is, if **ct** is the ciphertext encoding the symbols, we have the following:

$$\mathbf{CtS} \circ \mathbf{ModRaise} \circ \mathbf{StC}(\mathbf{ct}) = \mathbf{ct}_{\text{Re}}, \mathbf{ct}_{\text{Im}}$$

where $\mathbf{ct}_{\text{Re}}$ encrypts a plaintext vector $\mathbf{m}_{\text{Re}}$ such that $\mathbf{m}_{\text{Re}} \pmod{t} \in \{0, 1/2\}^{N/2}$, while $\mathbf{ct}_{\text{Im}}$ encrypts a plaintext vector $\mathbf{m}_{\text{Im}}$ such that $\mathbf{m}_{\text{Im}} \pmod{t} \in \{0, 1\}^{N/2}$.

Now, we perform the remaining bootstrapping procedure on the two ciphertexts as follows:

$$\begin{aligned} \text{ct}_{\text{Re}} &\leftarrow \text{LUT}_{h_1} \circ \text{EvalExp}(\text{ct}_{\text{Re}}) \\ \text{ct}_{\text{Im}} &\leftarrow \text{LUT}_{h_2} \circ \text{EvalExp}(\text{ct}_{\text{Im}}) \\ \text{ct} &\leftarrow \text{ct}_{\text{Re}} + i \cdot \text{ct}_{\text{Im}} \end{aligned}$$

where h_1 and h_2 are first-order Hermite interpolation polynomials that map the interpolation nodes (which are complex roots of unity) to integer values.²

Although this process requires two calls to both **LUT** and **EvalExp**, the associated complexity is minimal. The complexity of the **LUT** is negligible as it only involves two interpolation points. Furthermore, the two calls to **EvalExp** are effectively offset by our simultaneous evaluation of sine and cosine (See Appendix C.1).

This approach is also more level-efficient. Whereas a conventional three-point interpolation consumes $\lceil \log(3 \times 2) \rceil = 3$ levels, our two-point method requires only $\lceil \log(2 \times 2) \rceil = 2$ levels. Additionally, the error cleaning effect is enhanced because the procedure involves interpolating two points on a line rather than three points on a plane.

Cleaning without Bootstrapping The aforementioned approach can also be applied when cleaning must be performed without bootstrapping. Specifically, this involves separating the real and imaginary parts via a conjugation operation, as follows:

$$\text{ct}_{\text{Re}} = \text{ct} + \text{Conj}(\text{ct}), \quad \text{ct}_{\text{Im}} = \text{ct} - \text{Conj}(\text{ct})$$

where ct_{Re} and ct_{Im} are ciphertexts encrypting the original plaintext messages, each scaled by a factor of 2.

The cleaning process is then completed by evaluating a Hermite interpolation polynomial for the map $x \mapsto x/2$, and subsequently combining the real and imaginary parts.

4 Arbitrary Modular Arithmetic

Our radix scheme stores large integers by decomposing them into radices across slots, performs arithmetic operations, and then restores them to their unique radix representation. This scheme can be regarded as a homomorphic variant of hardware operations with generalized limb sizes, rather than being limited to binary representations. In this section, we present subtraction and comparison operations that allow hardware reduction algorithms such as Montgomery and Barrett to be implemented homomorphically. With these operations, our radix-based scheme can support modular arithmetic over an arbitrary modulus, just as the RNS in [6].

² Alternatively, the composition $\text{LUT} \circ \text{EvalExp}$ can be performed similarly to binary bootstrapping like [3].

4.1 Subtraction and Comparison

We can perform comparison by subtraction. Let $a, b \in \mathbb{Z}_{B^k}$ and let $c = a - b \pmod{B^k}$. The integer result $a - b$ can be expressed as $c + m \cdot B^k$, where the multiplier m encodes the sign. If $a \geq b$, then $m = 0$; otherwise, $m = -1$. In our radix-based scheme, this multiplier m corresponds to the value in the k -th slot after carry propagation. Therefore, if the k -th slot is 0, then $a \geq b$ is true, and if it is -1 , then $a \geq b$ is false.

However, a standard carry algorithm is insufficient because subtraction creates negative digits, and our carry procedure, **LToC**, does not propagate carries sequentially. To address this, we must modify the algorithm to correctly handle borrows. The logic can be illustrated with a simple two-digit example. Let $z = z_0 + z_1 B$ be the result of a subtraction. The digits z_0, z_1 will lie in the range $(-B, B)$. If z_0 is negative, it must be normalized by borrowing from the next position, rewriting the expression as $z = (z_0 + B) + (z_1 - 1)B$. This shows that a negative digit effectively propagates a carry of -1 (a borrow). Based on this principle, we introduce the following modifications to **LToC**:

- **Line 1:** The Look-up Table function ϕ is replaced by ϕ^* where $\phi^*(\cdot) : \mathbb{Z}_{2B-1} \rightarrow \{0, 1, 2\}$ where $\{0\} \rightarrow 1/2, [1, B) \rightarrow 0, [B, 2B-1) \rightarrow i$
- **Line 7:** The output update rule is changed to correctly subtract the propagated borrow: $\text{ct}_{\text{out}} \leftarrow \text{ct} + B \cdot \text{ct}_u - \rho_{-1}(\text{ct}_u)$.

We name this modified procedure **LToCneg**. It is a generalized carry algorithm that restores a subtraction result on $\mathbb{Z}_{B^k}$ to a unique digit representation, storing the sign of the result in the k th digit. In this case, since ϕ^* maps the symbol 0 to $1/2$, we add a 1 to the k -th digit to prevent unintended propagation. A subsequent rotation moving this sign to the 0th position yields the comparison result: a representation where the 0th digit is 1 or 0 and all others are zero. This result can then be used to perform a conditional subtraction. The pseudocode for these operations using **LToCneg** is presented in Algorithm 6.

4.2 Homomorphic Modular Reduction

Montgomery Reduction In general, when performing Montgomery reduction for a k -bit integer N , the intermediate value may grow up to $2k$ bits, and thus must be stored. Similarly, in our scheme with base B , reducing a modulus of size B^k requires using $2k$ instead of k as the base length parameter.

Now assume that $R = B^k$ satisfies the conditions for Montgomery parameters with respect to the target modulus N (e.g., $\gcd(N, B) = 1$ and $R > N$). If a ciphertext ct encodes a unique digit representation, the operation $[\cdot]_R$ can be realized as multiplication by an appropriate mask vector, while bit shifts such as $1/R$ can be performed through rotation operations.

Finally, since the previous section established that conditional subtraction can be performed homomorphically, the Montgomery reduction procedure can be carried out entirely within the homomorphic setting. The homomorphic Montgomery reduction algorithm **MontREDC** is described in Algorithm 7.

Algorithm 6 Comp, Sub, CondSub

Input: $\text{ct}_i = \text{Enc} \circ \text{REcd}(\mathbf{y}_i) \in \mathcal{R}_q^2$, for $i = 1, 2$. $\mathbf{v}$ is a vector with only the k -th digit being 1 and the rest being 0.

Output: $\text{ct}_{\text{comp}} = \text{Enc} \circ \text{REcd}(\mathbf{y}_1 \geq \mathbf{y}_2)$ or $\text{ct}_{\text{sub}} = \text{Enc} \circ \text{REcd}([\mathbf{y}_1 - \mathbf{y}_2]_{B^k}) \in \mathcal{R}_Q^2$ or $\text{ct}_{\text{out}} = \text{Enc} \circ \text{REcd}((\mathbf{y}_1 \geq \mathbf{y}_2) ? \mathbf{y}_1 - \mathbf{y}_2 : \mathbf{y}_1) \in \mathcal{R}_Q^2$

- 1: $\text{ct} \leftarrow \text{ct}_1 - \text{ct}_2 + \mathbf{v}$
- 2: $\text{ct}_{\text{sub}} \leftarrow \text{LCtoCneg}(\text{ct})$ ▷ Sub procedure returns ct_{sub}
- 3: $\text{ct}'' \leftarrow \text{ct}' \odot \mathbf{v}$
- 4: $\text{ct}_{\text{comp}} \leftarrow \rho_k(\text{ct}'')$ ▷ Comp procedure returns ct_{comp}
- 5: **for** $i = 0$ to $\log(k)$ **do**
- 6: $\text{ct}_{\text{tmp}} \leftarrow \rho_{-2^i}(\text{ct}_{\text{comp}})$
- 7: $\text{ct}_{\text{comp}} \leftarrow \text{ct}_{\text{comp}} + \text{ct}_{\text{tmp}}$
- 8: **end for**
- 9: $\text{ct}_{\text{out}} \leftarrow \text{ct}_{\text{comp}} \odot \text{ct}_{\text{sub}} + (1 - \text{ct}_{\text{comp}}) \odot \text{ct}_1$
- 10: **return** ct_{out}

Algorithm 7 MontREDC

Setting: $\mathbf{N}$ is the target modulus. $R = B^k$ such that $\gcd(R, \mathbf{N}) = 1$ and $R > \mathbf{N}$. $\mathbf{N}' = -\mathbf{N}^{-1} \bmod R$. Digit length parameter is set to $2k$ for the radix system.

Input: $\text{ct}_{\mathbf{T}} = \text{Enc} \circ \text{REcd}(\mathbf{y}) \in \mathcal{R}_q^2$, where $\|\mathbf{y}\|_{\infty} < \mathbf{N} \cdot R$. $\mathbf{v}$ is a masking vector that sets only the first $k/4$ slots to 1 and the others to 0.

Output: $\text{ct}_{\text{out}} = \text{Enc} \circ \text{REcd}([\mathbf{y} \cdot R^{-1}]_{\mathbf{N}}) \in \mathcal{R}_Q^2$

- 1: $\text{ct}_{T_{\text{mod}}} \leftarrow \text{ct}_{\mathbf{T}} \odot \mathbf{v}$
- 2: $\text{ct}_m \leftarrow \text{ExactMult}(\text{ct}_{T_{\text{mod}}}, \mathbf{N}')$
- 3: $\text{ct}_m \leftarrow \text{ct}_m \odot \mathbf{v}$
- 4: $\text{ct}_{mN} \leftarrow \text{ExactMult}(\text{ct}_m, \mathbf{N})$
- 5: $\text{ct} \leftarrow \rho_k(\text{ct}_{\mathbf{T}} + \text{ct}_{mN})$
- 6: $\text{ct}_{\text{out}} \leftarrow \text{CondSub}(\text{ct}, \mathbf{N})$
- 7: **return** ct_{out}

Base Reduction While Montgomery reduction is a general-purpose technique applicable to any modulus (with a proper choice of R and B), a modulus of special form, such as a generalized Mersenne number (GMN), can be reduced more efficiently by exploiting its algebraic structure. For example, for $p = 2^{255} - 19$ and $0 \leq x < p^2$, write $x = x_h 2^{255} + x_{\ell}$. Using $2^{255} \equiv 19 \pmod{p}$, set

$$x' := 19x_h + x_{\ell} \equiv x \pmod{p},$$

and since $0 \leq x' < 20p$, at most 5 conditional subtractions reduce x' into $[0, p)$. Folding once more, write $x' = x'_h 2^{255} + x'_{\ell}$ and set

$$x'' := 19x'_h + x'_{\ell} \equiv x \pmod{p}.$$

Then $0 \leq x'' < 2p$, so a single conditional subtraction suffices.

To generalize this method for a target modulus $\mathbf{N}$, we homomorphically evaluate the following:

$$\begin{aligned} x &= \sum_{0 \leq i < 2k} x_i \cdot B^i = \sum_{0 \leq i < k} x_i \cdot B^i + \sum_{k \leq i < 2k} x_i \cdot B^i \\ &\longrightarrow \sum_{0 \leq i < k} x_i \cdot B^i + \sum_{0 \leq i < k} x_{i+k} \cdot [B^{i+k}]_{\mathbf{N}} = \sum_{0 \leq i < k} c_i \cdot B^i, \end{aligned}$$

where

$$c_i = x_i + \sum_{0 \leq j < k} x_{j+k} \cdot D_{B,k}([B^{j+k}]_{\mathbf{N}})[i].$$

Equivalently, in vector notation, we can write

$$\mathbf{x}' = (x_0 \cdots x_{k-1}) + (x_k \cdots x_{2k-1}) \cdot T_{\mathbf{N}},$$

where

$$T_{\mathbf{N}} = [D_{B,k}([B^k]_{\mathbf{N}}) \mid \cdots \mid D_{B,k}([B^{2k-1}]_{\mathbf{N}})]^{\top}.$$

Therefore, by precomputing $T_{\mathbf{N}}$ and applying it as a homomorphic linear transformation, we can significantly reduce the size of the values. We refer to this algorithm as **baseREDC**, which is presented in Algorithm 8. The following theorem provides a worst-case upper bound on how much the input is reduced through **baseREDC**, in the absence of knowledge about the input values. The proof is provided in the Appendix B.4

Theorem 4. *Let $\mathbf{N} < B^k$ and $d = \lceil \log_B \mathbf{N} \rceil$. For $\mathbf{N} \leq \mathbf{M} < \mathbf{N}^2$ and $0 \leq x < \mathbf{M}$, write*

$$x = \sum_{i=0}^{k-1} x_i B^i + \sum_{i=k}^{k+d'} x_i B^i, \quad \mathbf{M} = \sum_{i=0}^d m_i B^i$$

for some $k \leq d' < d$.

Define

$$\mathbf{x}_{\ell} = (x_0, \dots, x_{k-1}), \quad \mathbf{x}_h = (x_k, \dots, x_{d'}, 0, \dots, 0),$$

$$\mathbf{v}_m = (B-1, \dots, B-1, m_d, 0, \dots, 0).$$

Then

$$x' = D_{B,k}^{-1}(\mathbf{x}_{\ell} + \mathbf{x}_h T_{\mathbf{N}}) \equiv x \pmod{\mathbf{N}},$$

and moreover

$$x' < d_m = B^k + D_{B,k}^{-1}(\mathbf{v}_m \cdot T_{\mathbf{N}}) < \mathbf{M}.$$

Algorithm 8 BaseREDC

Setting: $N < B^k$ is the target modulus. Digit length parameter is set to $2k$ for the radix scheme. T_N is the base reduction matrix and iteration number is β .
Input: $\text{ct} = \text{Enc} \circ \text{REcd}(\mathbf{y}) \in \mathcal{R}_q^2$, where $\|\mathbf{y}\|_\infty < N^2$
Output: $\text{ct}_{\text{out}} = \text{Enc} \circ \text{REcd}([\mathbf{y}]_N) \in \mathcal{R}_Q^2$

- 1: $\text{ct}_{\text{REDC}} \leftarrow \text{ct}$
- 2: $M \leftarrow N^2$
- 3: **for** $j = 0$ to β **do**
- 4: $\text{ct}' \leftarrow \text{BaseReduction}(T_N, \text{ct}_{\text{REDC}})$
- 5: **for** $i = 0$ to t **do**
- 6: $\text{ct}' \leftarrow \text{LC}(\text{ct}')$
- 7: **end for**
- 8: $\text{ct}_{\text{REDC}} \leftarrow \text{LCtoC}(\text{ct}')$
- 9: $M \leftarrow B^k + D_{B,k}^{-1}(\mathbf{v}_m \cdot T_N)$
- 10: Update $\mathbf{v}_m$ for M
- 11: $d_m \leftarrow \lfloor \log(M/N) \rfloor$
- 12: **end for**
- 13: **for** $i = 0$ to d_m **do**
- 14: $\text{ct}_{\text{out}} \leftarrow \text{CondSub}(\text{ct}_{\text{REDC}}, 2^{d_m-1-i} \cdot N)$
- 15: **end for**
- 16: **return** ct_{out}

5 Experiments

In this section, we present the numerical results of the proposed radix-based large-integer CKKS scheme. We provide proof-of-concept implementations³ for our method. All experiments were conducted on a single thread of an AMD Ryzen 9 7900X 12-Core Processor with 125 GB RAM, running Ubuntu 20.04, using Lattigo [1], a representative RNS-CKKS library.

Unless otherwise specified, we set $B = 2^4$ for all experiments. The detailed CKKS parameters used in the implementation are summarized in Table 4. An additional modulus was appended to the base moduli as a safeguard against overflow: for large k , the digit values after PolyMult become excessively large, which could lead to message overflow during the StC process. Although this safeguard is not strictly required in every setting, we consistently included it in all experiments. Furthermore, instead of denoting the number of moduli used in circuit depth by t , we express the circuit depth in terms of the $\log(PQ)$ range, thereby ensuring that all experiments are conducted under 128-bit security. When relevant, the value of t will also be explicitly indicated in the experimental results. Finally, we performed the cleaning using a first-order Hermite interpolation function.

Large Integer Multiplication We evaluated large integer multiplication from 16 bits up to 2048 bits using the proposed LazyMult and ExactMult algorithms.

³ https://github.com/Yuria20/radix_CKKS

$\log N$	(H, h)	$\log(PQ)$	Base	StC	CircuitDepth	LUT	Eval	CtS	P
16	$(N/2, 32)$	1618-1714	52+48	48×3	$48 \times (t+1) + 55 \times 2$	48×6	52×8	52×3	52×5

Table 4. The CKKS Scheme parameter set used for the experiments. Here N denotes the ring dimension, QP denotes the maximum key switching modulus, H, h denote the dense and spare Hamming weight for the sparse secret encapsulation [7], respectively, and Q and P denote the evaluation modulus and auxiliary modulus, respectively. When written as XY , it denotes having Y many X bit moduli. In **CircuitDepth**, 52×2 is the modulus for **HomDFT**.

(B, k)	#Slots	#Btp	LazyMult (Lat./Amort.)	ExactMult (Lat./Amort.)	Noise size
$(2^4, 4)$	4096	2+1	25.81 sec / 6.29 ms	39.89 sec / 9.73 ms	-19.74
$(2^4, 8)$	2048	2+1	25.71 sec / 12.55 ms	39.78 sec / 19.42 ms	-19.42
$(2^4, 16)$	1024	2+1	26.16 sec / 25.55 ms	40.30 sec / 39.35 ms	-17.62
$(2^4, 32)$	512	3+1	42.18 sec / 82.39 ms	57.41 sec / 112.14 ms	-17.63
$(2^4, 64)$	256	3+2	44.31 sec / 173.11 ms	74.76 sec / 292.04 ms	-18.83
$(2^4, 128)$	128	3+2	47.31 sec / 369.61 ms	78.11 sec / 610.26 ms	-18.08
$(2^4, 256)$	64	3+2	51.01 sec / 797.09 ms	81.24 sec / 1.26 sec	-17.23
$(2^4, 512)$	32	4+2	77.33 sec / 2.41 sec	109.02 sec / 3.41 sec	-17.66

Table 5. Timing for large integer multiplication. $\log B^k$ is modulus size and #Btp refers to the number of bootstrapping iterations, where the first term is the number of LC iterations t , and the second term is the number of diBtp for mapping symbol and error cleaning. Noise size is maximum noise.

The detailed timing performance is presented in Table 5. The correctness of the carry was verified by checking if the rounded result of the real part after decryption was in the range $[0, B)$.

Modular Multiplication We conducted experiments on homomorphic modular multiplication for RSA-2048 and elliptic-curve moduli such as $p = 2^{255} - 19$ (Curve25519) and $p = 2^{384} - 2^{128} - 2^{96} + 2^{32} - 1$ (P-384). For the RSA modulus, Montgomery reduction (MontREDC) was employed. For Curve25519, we applied a simple folding method, while for P-384 we used baseREDC.

In the case of Curve25519 and P-384, we report both *lazy* reductions (where certain conditional operations are omitted) and exact reductions (where all conditional steps are applied). For the Lazy method in both cases, the timings for the final LCtoC and CondSub operations were not included.

The folding for Curve25519 is based on the congruence relation $2^{256} \equiv 38 \pmod{2^{255} - 19}$. The baseREDC for P-384 was implemented with $\beta = 2$ iterations, where d_m is 7 in the first iteration and 1 in the second.

5.1 Comparison with Prior Works for High Precision HE

We compare our work with existing research on high-precision HE, focusing in particular on [6,24,25,28]. Detailed comparison results are summarized in Tables 1, 2, 6, 7.

Modulus	(B, k, t)	#Slots	Latency	Amortized time	Noise
Curve25519	$(2^4, 256, 3)$	128	172.34 sec	1.34 sec	-14.8
			244.41 sec	1.91 sec	
P-384	$(2^4, 512, 3)$	64	213.01 sec	3.33 sec	-14.2
			274.31 sec	4.28 sec	
RSA	$(2^4, 4096, 4)$	16	440.51 sec	27.53 sec	-13.3

Table 6. Modular multiplication time for specific moduli: Curve25519, P-384, and the 2048-bit RSA modulus. Gray-shaded cells indicate the results obtained with lazy reduction, where the output is not fully reduced modulo p , but instead lies within the range $[0, 2p)$.

Compared to TFHE-rs [28]⁴, our scheme demonstrates consistent superiority across most performance metrics except latency. For 64-bit integer multiplication, TFHE-rs is about $2\times$ faster in terms of latency, but our method achieves an improvement of approximately $498\times$ in amortized time. In particular, for 256-bit integers, our approach is about $4\times$ faster in terms of latency and shows an improvement of approximately $1059\times$ in amortized time. Moreover, as the bit width increases, our approach continues to widen its relative advantage in latency.

The main difference between our radix scheme and the nested RNS proposed in [6] is that our scheme performs *exact reduction*, whereas the RNS does not. Without exact reduction, additional functionalities such as comparison operations and LUT evaluations cannot be supported.

Another difference lies in the type of modulus each scheme favors. Our scheme is optimized for moduli of the form B^k , and can perform reduction for arbitrary moduli on top of this structure. In contrast, the nested RNS is more suitable for smooth-number moduli. Thus, if the target modulus is a smooth number representable in RNS form, the nested RNS is highly efficient. However, such smooth-number moduli rarely appear in real applications. Moreover, if the modulus is not smooth but still fits within a single RNS layer, the nested RNS can still outperform our method. Nevertheless, when considering real-world application, our scheme provides more practical and decisive advantages.

Furthermore, in the context of homomorphic signing, our scheme also demonstrates superior performance compared to [6].

For instance, in Curve-25519, our scheme is about $1.4\times$ faster in latency and achieves about $5.6\times$ improvement in amortized time. For P-384, our scheme is about $1.15\times$ faster in latency and shows about $2.3\times$ improvement in amortized time. For RSA, [6] is about $1.39\times$ faster in latency, but our scheme is about $2.87\times$ faster in amortized time.

⁴ We used commit hash 97574bdae87962cc537d91c16d53a008c0423c29 and measured the performance of the `integer-bench`, specifically the `mul_parallelized` benchmark, under a single-threaded setting.

Scheme		$\log(t)$	# slots	Latency	Amortized time
DM/CGGI	TFHE-rs [28]	64	1	19.6 sec	19.6 sec
		128	1	77.5 sec	77.5 sec
		256	1	309 sec	309 sec
CKKS	[6]	$\approx_s 128$	256	30.5 sec	119 ms
		$\approx_s 256$	128	30.6 sec	239 ms
		256	32	242 sec	7.5 sec
		2048	4	316 sec	79 sec
	Ours	64	1024	40.3 sec	39.35 ms
		128	512	57.4 sec	112.14 ms
		256	256	74.8 sec	292.04 ms
		2048	32	109 sec	3.41 sec
	[24]	64	16384	240.8 sec	14.69 ms
		256	32768	1764 sec	53.83 ms
	[25]	64	2048	41.25 sec	20.14 ms
		128	1024	50.91 sec	49.72 ms
		256	512	52.79 sec	103.09 ms
		2048	64	62.2 sec	970.72 ms
BFV/BGV	[27]	≈ 168	≤ 65536	128 sec	≥ 1.95 ms
		≈ 234	≤ 131072	392 sec	≥ 2.99 ms

Table 7. Latency and amortized time comparison with TFHE-rs, [6,24,25,27]. Here, $\approx$ denotes a nearby value, while $\approx_s$ denotes a smooth number close to $\log(t)$. Timings for [27] are taken from Table 4 in [27], under a similar experimental setup.

Compared to [24]⁵, our scheme achieves significantly lower latency, though with small number of data. This trade-off arises from the fact that [24] requires $O(k)$ bootstrapping operations with respect to the bit-size k , whereas our scheme requires only $O(\log k)$ operations. As a result, the latency advantage of our scheme becomes more pronounced as the bit width increases, making it more suitable for high-precision applications.

We compare our LazyMult algorithm with the radix-based scheme proposed in [25].⁶ In terms of latency, our algorithm exhibits a slight advantage at lower bit-widths (64, 128, and 256 bits), as it necessitates one to two fewer bootstrapping operations. However, as the bit size increases, the scheme from [25] becomes marginally superior. This shift is attributed to the overhead from our HomDFT operations, which eventually outweighs the savings from the reduced number of bootstrappings. Regarding amortized time, the approach in [25] is approximately twice as efficient. This performance difference stems from the data encoding strategy: our method stores digits in slots, whereas [25] embeds them directly into the ring coefficients.

Methodologically, the schemes also diverge. The work in [25] employs the CinS packing [22], which leverages the structural similarity between the CKKS encoding matrix and DFT. This enables a partially decomposed encoding matrix to

⁵ While [24] used $\log N = 15$, our results are based on $\log N = 16$.

⁶ [25] did not provide experimental results on exact digit carrying.

naturally implement polynomial arithmetic in $(\mathbb{Z}[X]/(X^{2k}+1))^{N/4k}$. In contrast, our scheme relies on DFT transformations to implement polynomial arithmetic. While CinS packing avoids the overhead of DFT, it’s important to note that this also permutes the order of the slots into a bit-reversed. Despite these similarities in encoding, the reduction and carry algorithms differ fundamentally. Our LC algorithm always propagates the quotient to the next digit, whereas the method of [25] accumulates the least significant bit of B in each loop. Moreover, according to [25], their scheme requires $O(k) = t + (\log_B(t) + 1)(k - 1) + 1$ discrete bootstrapping operations for parameter $t < k$, which further increases the overall computational cost.

Compared to state-of-the-art RLWE-based exact schemes such as BFV/BGV [27], our method provides lower latency. In contrast, the amortized efficiency of BFV/BGV depends on the number of available slots, resulting in a trade-off between latency and amortized time. As the plaintext bit-width increases, BFV/BGV must increase the ring dimension, which leads to higher latency. To the best of our knowledge, the largest reported plaintext size for BFV/BGV is 234 bits (in [27]), and supporting larger plaintext sizes is expected to incur substantial overhead.

5.2 Limitations and Feasible Optimization

Our radix scheme can support precision up to $B^{N/4}$ when choosing $k = N/4$. For instance, with the widely adopted ring dimension $N = 2^{16}$ and $B = 2^8$, this corresponds to a precision of about 131,072 bits. However, the complexity of the HomDFT required for polynomial multiplication grows with k , and in practice, when $k \geq 2048$, the HomDFT cost becomes almost identical to the bootstrapping latency.

To mitigate the complexity of HomDFT, several alternative optimization strategies can be considered. For example, one may employ the Cooley-Tukey algorithm or apply CinS packing [22] followed by multiplication with an inverse bit-reversal matrix. The former, however, may reduce the security of ring LWE due to increased levels (for $N = 2^{16}$), which requires careful adjustment of the modulus size. In the latter case, since operations are performed over subrings rather than individual slots, the algorithms proposed in this paper may not be directly applicable. However, we do not rule out the possibility of applying them; rather, a careful analysis is required to understand how radix-based carry propagation and conditional operations can be interpreted and implemented at the subring level. As such, this approach presents a promising direction for future research.

To further extend precision, one may also consider a hybrid RNS–Radix scheme. For example, [6] designs a homomorphic RNS by applying the modular reduction of [26] differently for each slot, and similarly, it is possible to assign ℓ different radix bases $\{B_i\}_{0 \leq i < \ell}$ to each slot in our scheme. However, homomorphic base conversion in such a hybrid scheme requires reduction across distinct coprimes represented in radix form, and is therefore unlikely to be more efficient than the method proposed in [6].

Ultimately, this work does not merely pursue optimization-based speedups, but highlights the practical feasibility of using CKKS to replace DM/CGGI in general-purpose settings. Our radix scheme supports flexible precision, suggesting strong potential for scalable and versatile HE schemes.

Acknowledgments. This work was supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No.RS-2024-00399401, Development of post-quantum security infrastructure transition and comprehensive quantum security verification technology, 50% & No. RS-2024-00399491, Development of Privacy-Preserving Multiparty Computation Techniques for Secure Multiparty Data Integration, 50%).

References

1. Lattigo v6. Online: <https://github.com/tuneinsight/lattigo> (Aug 2024), ePFL-LDS, Tune Insight SA
2. Alexandru, A., Kim, A., Polyakov, Y.: General functional bootstrapping using CKKS. In: *Advances in Cryptology – CRYPTO 2025*. pp. 304–337 (2025). https://doi.org/10.1007/978-3-032-01881-6_10
3. Bae, Y., Cheon, J.H., Kim, J., Stehlé, D.: Bootstrapping bits with ckks. In: *Advances in Cryptology – EUROCRYPT 2024*. pp. 94–123 (2024). https://doi.org/10.1007/978-3-031-58723-8_4
4. Bae, Y., Kim, J., Stehlé, D., Suvanto, E.: Bootstrapping small integers with ckks. In: *Advances in Cryptology – ASIACRYPT 2024*. pp. 330–360 (2025). https://doi.org/10.1007/978-981-96-0875-1_11
5. Bergerat, L., Boudi, A., Bourgerie, Q., Chillotti, I., Ligier, D., Orfila, J.B., Tap, S.: Parameter optimization and larger precision for (t)fhe. *Journal of Cryptology* **36**(3), 521–585 (2023). <https://doi.org/10.1007/s00145-023-09463-5>
6. Boneh, D., Kim, J.: Homomorphic encryption for large integers from nested residue number systems. In: *Advances in Cryptology – CRYPTO 2025*. pp. 338–370 (2025). https://doi.org/10.1007/978-3-032-01881-6_11
7. Bossuat, J.P., Troncoso-Pastoriza, J., Hubaux, J.P.: Bootstrapping for approximate homomorphic encryption with negligible failure-probability by using sparse-secret encapsulation. In: *Applied Cryptography and Network Security (ACNS 2022)*. pp. 521–541. Springer (2022). https://doi.org/10.1007/978-3-031-09234-3_26
8. Brakerski, Z.: Fully homomorphic encryption without modulus switching from classical gapsvp. In: *Advances in Cryptology – CRYPTO 2012*. pp. 868–886 (2012). https://doi.org/10.1007/978-3-642-32009-5_50
9. Brakerski, Z., Gentry, C., Vaikuntanathan, V.: (leveled) fully homomorphic encryption without bootstrapping. *ACM Trans. Comput. Theory* **6**(3) (Jul 2014). <https://doi.org/10.1145/2633600>
10. Chartier, P., Koskas, M., Lemou, M., Méhats, F.: Fully homomorphic encryption on large integers. *Cryptology ePrint Archive*, Paper 2024/155 (2024), <https://eprint.iacr.org/2024/155>
11. Chartier, P., Koskas, M., Lemou, M., Méhats, F.: Homomorphic sign evaluation with a rns representation of integers. In: *Advances in Cryptology – ASIACRYPT 2024*. pp. 271–296 (2025). https://doi.org/10.1007/978-981-96-0875-1_9

12. Chen, H., Laine, K., Player, R., Xia, Y.: High-precision arithmetic in homomorphic encryption. In: Topics in Cryptology – CT-RSA 2018: The Cryptographers’ Track at the RSA Conference 2018, San Francisco, CA, USA, April 16–20, 2018, Proceedings. pp. 116–136 (2018). https://doi.org/10.1007/978-3-319-76953-0_7
13. Cheon, J.H., Kim, A., Kim, M., Song, Y.: Homomorphic encryption for arithmetic of approximate numbers. In: Advances in Cryptology – ASIACRYPT 2017. pp. 409–437 (2017). https://doi.org/10.1007/978-3-319-70694-8_15
14. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: Tfhe: Fast fully homomorphic encryption over the torus. *Journal of Cryptology* **33**(1), 34–91 (2020). <https://doi.org/10.1007/s00145-019-09319-x>
15. Chillotti, I., Ligier, D., Orfila, J.B., Tap, S.: Improved programmable bootstrapping with larger precision and efficient arithmetic circuits for tfhe. In: Advances in Cryptology – ASIACRYPT 2021. pp. 670–699 (2021). https://doi.org/10.1007/978-3-030-92078-4_23
16. Drucker, N., Moshkovich, G., Pelleg, T., Shaul, H.: Bleach: Cleaning errors in discrete computations over ckks. *Journal of Cryptology* **37**(1), 3 (2024). <https://doi.org/10.1007/s00145-023-09483-1>
17. Dumezy, J., Alexandru, A., Polyakov, Y., Clet, P.E., Chakraborty, O., Boudguiga, A.: Evaluating larger lookup tables using CKKS. *IACR Transactions on Cryptographic Hardware and Embedded Systems* **2026** (2026)
18. Fan, J., Vercauteren, F.: Somewhat practical fully homomorphic encryption. *Cryptology ePrint Archive*, Paper 2012/144 (2012), <https://eprint.iacr.org/2012/144>
19. Geelen, R., Vercauteren, F.: Fully homomorphic encryption for cyclotomic prime moduli. In: Advances in Cryptology – EUROCRYPT 2025: 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Madrid, Spain, May 4–8, 2025, Proceedings, Part III. pp. 366–397 (2025). https://doi.org/10.1007/978-3-031-91131-6_13
20. Gentry, C.: Fully homomorphic encryption using ideal lattices. In: Proceedings of the 41st Annual ACM Symposium on Theory of Computing (STOC). pp. 169–178 (2009). <https://doi.org/10.1145/1536414.1536440>
21. Halevi, S., Polyakov, Y., Shoup, V.: An improved rns variant of the BFV homomorphic encryption scheme. In: Topics in Cryptology – CT-RSA 2019. pp. 83–105 (2019). https://doi.org/10.1007/978-3-030-12612-4_5
22. Ju, J.H., Park, J., Kim, J., Kang, M., Kim, D., Cheon, J.H., Ahn, J.H.: Neujeans: Private neural network inference with joint optimization of convolution and fhe bootstrapping. In: Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS). pp. 4361–4375 (2024). <https://doi.org/10.1145/3658644.3690375>
23. Kim, J.: Bootstrapping GBFV with CKKS. *Cryptology ePrint Archive*, Paper 2025/888 (2025), <https://eprint.iacr.org/2025/888>
24. Kim, J.: Efficient homomorphic integer computer from ckks. *IACR Transactions on Cryptographic Hardware and Embedded Systems* **2025**, 873–898 (09 2025). <https://doi.org/10.46586/tches.v2025.i4.873-898>
25. Kim, J.: Faster homomorphic integer computer. *Cryptology ePrint Archive*, Paper 2025/1440 (2025), <https://eprint.iacr.org/2025/1440>
26. Kim, J., Noh, T.: Modular reduction in ckks. *IACR Communications in Cryptology* **2** (2025). <https://doi.org/10.62056/aeuvr-iuc>
27. Kim, J., Seo, J., Song, Y.: Simpler and faster bfv bootstrapping for arbitrary plaintext modulus from ckks. In: Proceedings of the 2024 ACM SIGSAC Conference

- on Computer and Communications Security (CCS). pp. 2535–2546 (2024). <https://doi.org/10.1145/3658644.3670302>
28. Zama: Tfhe-rs: A pure rust implementation of the tfhe scheme for boolean and integer arithmetics over encrypted data. <https://github.com/zama-ai/tfhe-rs> (2022)

A The CKKS Scheme

CKKS is a ring LWE-based HE scheme that approximates complex vector in $\mathbb{C}^{N/2}$. The plaintext space of CKKS is $\mathcal{R}$. Mapping complex vectors to elements of the $\mathcal{R}$ is called encoding, as described below.

Encoding Let $\zeta_j = \exp(2\pi i \cdot 5^j/2N)$ for $0 \leq j < N/2$. The map $m(X) \rightarrow (m(\zeta_j))_{0 \leq j < N/2} \in \mathbb{C}^{N/2}$ is an isomorphism between $\mathbb{R}[X]/(X^N + 1)$ and $\mathbb{C}^{N/2}$. It is also an isometric transformation that behaves like the discrete Fourier transform. Therefore, for convenience, we will refer to this mapping as DFT. Similarly, we can define the inverse transform of DFT as DFT^{-1} . Given a $\Delta > 0$, the encoding/decoding algorithm of CKKS is defined as follows.

$$\text{Ecd} : \mathbb{C}^{N/2} \rightarrow \mathcal{R} : \mathbf{z} \rightarrow \lfloor \Delta \cdot \text{DFT}^{-1}(\mathbf{z}) \rfloor \in \mathcal{R}.$$

$$\text{Dcd} : \mathcal{R} \rightarrow \mathbb{C}^{N/2} : m(X) \rightarrow \frac{1}{\Delta} \text{DFT}(m(X)) \in \mathbb{C}^{N/2}.$$

Encryption Given a plaintext $m \in \mathcal{R}$, a ciphertext of m is a pair $\text{ct} = (b, a) \in \mathcal{R}_Q^2$ such that $(b, a) \cdot (1, s) = m + e \pmod{Q}$ for a secret key $s \in \mathcal{R}$ and a small error $e \in \mathcal{R}$. Unlike other HE schemes, CKKS does not separate errors from plaintext, but manages the size of the error. Because of this difference, the CKKS scheme is called an approximate arithmetic scheme. The decryption algorithm is defined as

$$\text{Dec}(\text{ct}) = (b, a) \cdot (1, s) \in \mathcal{R}_Q^2.$$

Homomorphic Operations Given CKKS ciphertext $\text{ct}_1 = (b_1, a_1), \text{ct}_2 = (b_2, a_2) \in \mathcal{R}_Q^2$ and CKKS plaintext $m(X)$, the addition algorithm is defined as

$$\text{ct}_1 + \text{ct}_2 = (b_1 + b_2, a_1 + a_2) \in \mathcal{R}_Q^2.$$

Multiplication is not as simple as addition. First, the tensor product of two ciphertexts is defined as

$$\text{Tensor}(\text{ct}_1, \text{ct}_2) = \text{ct}_1 \otimes \text{ct}_2 = (d_0, d_1, d_2) = (b_1 b_2, b_1 a_2 + b_2 a_1, a_1 a_2) \in \mathcal{R}_Q^3.$$

The output of the tensor product increases the size of the ciphertext and requires $(1, s, s^2)$ as the decryption key instead of the existing decryption key $(1, s)$. The process of reducing the size of the ciphertext using a key switching procedure that moves the secret s^2 to s is called relinearization. Relinearization is defined as

$$\text{Relin}(d_0, d_1, d_2) = (d_0, d_1) + \text{KeySwitch}_{s^2 \rightarrow s}(d_2)$$

Since two ciphertexts with scaling factor Δ are multiplied, the scale of the ciphertext must be reduced to Δ . This procedure is called rescaling and is performed by

approximate division. Given a ciphertext $\text{ct} = (b, a) \in \mathcal{R}_Q^2$ and $q|Q$, the rescaling algorithm for q is defined as

$$\text{Rescale}(\text{ct}) = (\lfloor b/q \rfloor, \lfloor a/q \rfloor) \in \mathcal{R}_{Q/q}^2.$$

Finally, CKKS multiplication is performed as follows

$$\text{Mult}(\text{ct}_1, \text{ct}_2) = \text{Rescale} \circ \text{Relin} \circ \text{Tensor}(\text{ct}_1, \text{ct}_2)$$

We are interested in how much homomorphic multiplication increases the error. Let the plaintexts of the ciphertexts ct_1, ct_2 be $\Delta m_1 + e_1, \Delta m_2 + e_2$, respectively. The result message of Mult is as follows

$$\Delta m_1 m_2 + (m_1 e_2 + m_2 e_1) + e$$

where e is key switching and rescaling error. This means that if the message size is too large, the multiplication error will increase.

In CKKS, in addition to addition and multiplication, there is a *rotation* operation that cyclically shifts the slot vector. The k -shift rotation is defined as follows

$$\rho_k(\text{ct}) = (b(X^{5^k}), 0) + \text{KeySwitch}_{s(X^{5^k}) \rightarrow s}(a(X^{5^k}))$$

Bootstrapping CKKS multiplication consumes modulus, so if there are insufficient modulus resources, multiplication can no longer be performed. The procedure by which CKKS recovers modulus resources while maintaining message precision is called bootstrapping. We will treat the conventional bootstrapping algorithm as a black box and discuss discrete CKKS bootstrapping in detail in section 2.2.

B Proofs

B.1 Proof of Theorem 2 : Correctness of LC

Proof. Since the condition on ε_b is satisfied, the correctness of the discrete bootstrapping procedure guarantees that both the quotient and the remainder for each digit can be computed approximately.

It suffices to consider a single digit vector. For $\mathbf{z}_0 = (z_i)_{0 \leq i < 2k}$, after applying LC we obtain

$$\mathbf{z}'_0 = ([z_i]_B + Q_B(z_{i-1}))_{0 \leq i < k},$$

and the decoded value is preserved as shown below:

$$\begin{aligned} y_0 = D_{B,k}^{-1}(\mathbf{z}_0) &= \sum_{0 \leq i < 2k} z_i \cdot B^i \mod B^k \\ &= \sum_{0 \leq i < k} ([z_i]_B + B \cdot Q_B(z_i)) \cdot B^i \mod B^k \\ &= \sum_{0 \leq i < k} ([z_i]_B + Q_B(z_{i-1})) \cdot B^i \\ &= D_{B,k}^{-1}(\mathbf{z}'_0) \mod B^k. \end{aligned}$$

Finally, we obtain the bound

$$\|z'_0\|_\infty < Q_B(U_{\mathbf{z}}) + [U_{\mathbf{z}}]_B < O\left(\frac{U_{\mathbf{z}}}{B}\right) + B,$$

which completes the proof. $\square$

B.2 Proof of Theorem 3 : Correctness of LCtoC

Proof. The condition $\|\varepsilon\|_\infty < \varepsilon_b$ ensures the correctness of SIBoot_f . Let $\text{ct}' = \text{Enc}(\mathbf{z} + \varepsilon')$ with $\|\varepsilon'\|_\infty < \varepsilon_b$. According to the principles of binary carry behavior, if the i -th digit produces a carry, then the sequence of digits up to the i -th position must have a tail of the form $\{2, \{1\}^*\}$. In contrast, when the tail takes the form $\{0, \{1\}^*\}$ or when all digits up to the i -th position are ones, it corresponds to a non-carry state. Concretely, one would need to evaluate $f(z_0, \dots, f(z_{i-2}, f(z_{i-1}, z_i)))$ for all $0 \leq i < k$. Nevertheless, by Lemma 2, the carry status can be determined through $\log(k)$ -recursive evaluations of f .

We now turn to the analysis of the noise in the algorithm. First, consider the evaluation of f . Assume that the elements of $\mathbf{z}'$ are restricted to $\{0, 1/2, i\}$. Then,

$$\begin{aligned} |f(x + \varepsilon_x, y + \varepsilon_y) - f(x, y)| &= |(\varepsilon_x + \bar{\varepsilon}_x)(y - x) + (\varepsilon_y - \varepsilon_x)(x + \bar{x}) \\ &\quad + (\varepsilon_x + \bar{\varepsilon}_x)(\varepsilon_y - \varepsilon_x)| < \sqrt{5} \max\{\varepsilon_x, \varepsilon_y\}. \end{aligned}$$

Next, for the evaluation of τ , again restricting the inputs to $\{0, 1/2, i\}$, we have

$$|\tau(x + \varepsilon_x) - \tau(x)| = \left| \frac{i}{2} \cdot (\bar{\varepsilon}_x - \varepsilon_x) \right| < \varepsilon.$$

Finally, multiplication by B amplifies the error by a factor of B .

Furthermore, ignoring the error introduced by key switching, the evaluation of f for $\log(k)$ times, followed by one evaluation of τ , and finally a multiplication by B , results in an overall error growth bounded by $5^{\log(k)/2} \cdot B$. This provides an upper bound that guarantees the correctness of the algorithm. If this bound is not satisfied, correctness can still be ensured by applying a small number of error-cleaning procedures at intermediate steps. $\square$

Lemma 2. Let $f : \{a, b, c\}^2 \rightarrow \{a, b, c\}$ be a function defined by

$$f(x, y) = \begin{cases} a, & \text{if } y = a, \\ x, & \text{if } y = b, \\ c, & \text{if } y = c. \end{cases}$$

Let $\mathbf{x} \in \{a, b, c\}^k$ and $\mathbf{x}_{s,t} = (x_i)_{s \leq i < t}$. Define a family of functions $\{f_k\}_{k \geq 2}$ recursively by

$$f_2 = f, \quad f_k(\mathbf{x}_{0,k}) = f(x_0, f_{k-1}(\mathbf{x}_{1,k})) \quad \text{for } k \geq 3.$$

Similarly, define another family of functions $\{g_k\}_{k \geq 1}$ by

$$g_1 = f$$

$$g_k(\mathbf{x}_{0,2^k}) = f(g_{k-1}(\mathbf{x}_{0,2^{k-1}}), g_{k-1}(\mathbf{x}_{2^{k-1},2^k})) \quad \text{for } k \geq 2.$$

Then the two functions f_{2^k} and g_k are identical as functions; that is,

$$f_{2^k} = g_k \quad \text{as functions on } \{a, b, c\}^{2^k}.$$

Proof. It is trivial that $f_2 = g_1$. Assume that $f_{2^{k-1}} = g_{k-1}$ for some $k \geq 2$. Then

$$\begin{aligned} f_{2^k}(\mathbf{x}) &= f(x_0, f(x_1, \dots, f(x_{2^{k-1}-1}, f_{2^{k-1}}(\mathbf{x}_{2^{k-1},2^k})))) \\ &= f(x_0, f(x_1, \dots, f(x_{2^{k-1}-1}, g_{k-1}(\mathbf{x}_{2^{k-1},2^k})))) \end{aligned}$$

where the second equality follows from the induction hypothesis.

If $g_{k-1}(\mathbf{x}_{2^{k-1},2^k}) = b$, then by the definition of f ,

$$f_{2^k}(\mathbf{x}) = f(x_0, f(x_1, \dots, f(x_{2^{k-1}-2}, x_{2^{k-1}-1}))) = f_{2^{k-1}}(\mathbf{x}_{0,2^{k-1}}) = g_{k-1}(\mathbf{x}_{0,2^{k-1}}).$$

Otherwise, by the definition of f ,

$$f_{2^k}(\mathbf{x}) = g_k(\mathbf{x}_{2^{k-1},2^k}).$$

Finally, since

$$g_k(\mathbf{x}_{0,2^k}) = f(g_{k-1}(\mathbf{x}_{0,2^{k-1}}), g_{k-1}(\mathbf{x}_{2^{k-1},2^k}))$$

it follows that $f_{2^k}(\mathbf{x})$ and $g_k(\mathbf{x})$ produce identical outputs in all cases. Hence,

$$f_{2^k} = g_k \quad \text{as functions.}$$

B.3 Proof of Lemma 1

Proof. Let $\|\mathbf{z}\|_\infty \in [0, B^t]$. Let $\mathbf{z}^{(i)}$ be the result of iterating equation (1) i times.

If $\|\mathbf{z}^{(i)}\|_\infty \leq B^{t-i} + B - 2$, then

$$\|\mathbf{z}^{(i)}\|_\infty \leq Q_B(B^{t-i} + B - 2) + B - 1 = B^{t-(i+1)} + B - 2$$

Furthermore, since $\mathbf{z}^{(0)} \leq B^t + B$, it follows by mathematical induction that after applying (1) t times, the value of $\mathbf{z}$ is less than $2B - 1$.

Now, let's consider two polynomials, $z(X)$ and $z'(X)$, of degree less than k with digits bounded by $2B - 1$. We can conservatively estimate the upper bound of the digits as follows:

$$\|z(X) \cdot z'(X)\|_\infty \leq k(2B - 2)^2 = 4kB^2 - 8kB + 4k = O(kB^2)$$

According to the preceding statement, we can reduce the upper bound of the digits to $2B - 1$ using $O(\log_B(kB^2))$ bootstrappings (i.e., the LC algorithm). Therefore, the complexity for the number of bootstrappings is:

$$O(\log_B(kB^2)) = O(\log_B k) = O(\log k)$$

□

B.4 Proof of Theorem 4 : baseREDC

Proof. The congruence $x' \equiv x \pmod{N}$ follows directly, since $D_{B,k}^{-1}(\mathbf{x}_\ell + \mathbf{x}_h T_N)$ replaces each B^i for $k \leq i < d'$ with its reduction modulo N .

Assume that M is known; in particular, the highest nonzero digit m_d of M is given. Even without knowing the exact value of x , the d th digit cannot exceed m_d . Hence, an upper bound can be obtained by fixing the d th digit as m_d and setting all digits $0 \leq i < d$ to $B - 1$.

Equivalently,

$$x_h < \sum_{i=0}^{d-1} (B-1)B^i + m_d B^d.$$

Applying baseREDC to B^i for $k \leq i \leq d$ then yields

$$x' < B^k + D_{B,k}^{-1}(\mathbf{v}_m \cdot T_N).$$

□

C Bootstrapping and Cleaning

In this section, we provide a detailed description of the bootstrapping and cleaning used in our algorithm.

C.1 Simultaneous evaluation of cos and sin.

Recall that we have $N/2$ zero slots to avoid the cyclic shift phenomenon of the DFT. We leverage this to simultaneously evaluate the sin and cos functions required for the EvalExp function in diBtp, as follows:

$$\mathbf{z} = (z_1, \mathbf{0}) \rightarrow \mathbf{z}_{\text{exp}} = \text{EvalVecPoly}_{\text{exp}}(\mathbf{z} + \rho_{N/4}(\mathbf{z}))$$

where $\text{EvalVecPoly}_{\text{exp}}$ is a vectorized polynomial evaluation that computes $\cos(2\pi \cdot x/B)$ for indices i such that $0 < i < N/4$, and $\sin(2\pi \cdot x/B)$ otherwise.

We then perform the LUT_f step as follows:

$$\mathbf{z}_{\text{exp}} \rightarrow \text{EvalVecPoly}_{\text{LUT}}(\mathbf{z} + \rho_{N/4}(\mathbf{z}))$$

where $\text{EvalVecPoly}_{\text{LUT}}$ is a vectorized polynomial evaluation that computes $f(x)$ for indices i such that $0 < i < N/4$, and 0 otherwise.

In our actual implementation, the aforementioned method is not used for LCToCneg, as this algorithm stores a 1 in the k -th digit for each integer.

C.2 High-order Hermite Interpolation

In all our experiments, we employed first-order Hermite interpolation. A reader might then ask the following question: Why not use higher-order Hermite interpolation for the LCtoC algorithm, which is the most sensitive to error?

Although higher-order Hermite interpolation provides greater precision, it consumes additional multiplicative levels and has a higher latency than lower-order interpolation. In fact, with our experimental parameters (i.e., for first-order Hermite interpolation), the available depth at the moment error cleaning was required was less than the depth needed for the interpolation polynomial. Therefore, with the primary goal of minimizing latency, we set the ring dimension to $N = 2^{16}$ and did not use higher-order Hermite interpolation.

We note that our experimental parameters are not optimal, and clarify that it is possible to use higher-order Hermite interpolation by increasing the ring dimension or setting more optimal parameters.